

algoexpert 总结

two number of sum 两数之和

Valid Subsequence 合法子链

sorted Squared Array

tournament Winner

non Constructible Change

find Closest Value In Bst

branch Sums 分支求和

node Depths

depth First Search

mini mum WaitingTime

class Photos

tandem Bicycle

remove Duplicates From LinkedList

get Nth Fib

product Sum

Binary Search

Find Three Largest Numbers

Bubble Sort

Insertion Sort

Selection Sort

Is Palindrome

caesar Cipher Encryptor

run Length Encoding

generate Document

first Non Repeating Character

three Number Sum

smallest Difference

move Element To End

is Monotonic

spiral Traverse

longest Peak (最长的峰顶)

array Of Products (列的交叉乘积)

first Duplicate Value

merge Overlapping Intervals

BST Construction 二叉树的构建

validate Bst

二叉树的遍历【三种遍历的方法】

minHeight Bst【通过一个序列构建一个最小高度二叉树】

find KthLargestValue In Bst 【找到树里第K大的值】

reconstruct Bst【根据先序遍历构建二叉树】

invert Binary Tree

binary Tree Diameter

find Successor

height Balanced Binary Tree

max SubsetSum No Adjacent

number Of Ways To Make Change

min number Of Ways To Make Change

levenshtein Distance

number Of WaysTo Traverse Graph

kadanes Algorithm

Single Cycle

breadth First Search (层次遍历)

River Sizes

Youngest Common Ancestor

remove Islands

cycle In Graph

minimum Passes Of Matrix (邻里反转)

task Assignment (任务分配)

valid Starting City (适合起始的城市)

Min Heap Construction

Doubly Linked List

Remove Kth Node From End

Sum Of LinkedLists

Get Permutations

Powerset

Phone Number Mnemonics

Staircase Traversal

Search In Sorted Matrix

Three Number Sort

Min Max Stack (最大最小栈)

semordnilap (寻找倒装的数)

balanced Brackets (平衡的括号)

sunset Views (水平线视角【根据方向寻找逐步的最大值】)

sort Stack (只用pop和append 栈的方法解决排序问题)

next Greater Element (找到当前位置后面离它最近比当前值大的数)

longest Palindromic Substring

group Anagrams

valid IP Addresses

reverse Words In String

zero Sum Subarray

minimum Characters For Words

Suffix Trie

Merge Binary Trees

Stable Internships

Two Colorable

Zero Sum Subarray

MergingLinkedLists

SymmetricalTree

One Edit

Union Find

Four Number Sum

Subarray Sort

Subarray Sort

Min Rewards

ZigzagTraverse

Same Bsts

Validate Three Nodes

Max Path Sum In Binary Tree

Find Nodes Distance K

Max Sum Increasing Subsequence

Longest Common Subsequence

Count Squares

min Number Of Jumps

water Area

knapsack Problem

Disk Stacking

two number of sum 两数之和

总结：通过使用2个for循环来遍历两个列表，验证其的和是否等于target。

```
1 def twoNumberSum(array, targetSum):
2     # Write your code here.
3     a = len(array)
4     for i in range(a-1):
5         firstNum = array[i]
6         for j in range(i+1,a):
7             secondNum = array[j]
8             if firstNum+secondNum == targetSum:
9                 return [firstNum,secondNum]
```

Valid Subsequence 合法子链

总结：通过两个指针来分别指向母链和子链。

```
1 def isValidSubsequence(array, sequence):
2     # Write your code here.
3     arrId = 0
4     seqId = 0
5     while arrId < len(array) and seqId < len(sequence):
6         if array[arrId]==sequence[seqId]:
7             seqId+=1
8             arrId += 1
9     return seqId == len(sequence)
```

sorted Squared Array

总结：通过列表的append 来完成子链的排序和求和。

```
1 def sortedSquaredArray(array):
2     new_array = []
3     for each_element in array:
4         each_element *= each_element
5         new_array.append(each_element)
6     new_array.sort()
7     return new_array
```

tournament Winner

总结：首先我们必须明确更新的方式，确认和results列表相匹配的获胜队伍。

通过创建scores字典来更新实时的队伍分数。currentBestTeam 表示分数最高的队伍。

enumerate 给competitions 标上序号。

更新的方法，确定获胜队伍是否已经在字典中，不在创建获胜队伍的字典信息。如果已经在字典中就更新获胜队伍的字典信息。+3分。

Sample Input

```
competitions = [
    ["HTML", "C#"],
    ["C#", "Python"],
    ["Python", "HTML"],
]
results = [0, 0, 1]
```

Sample Output

```
"Python"
// C# beats HTML, Python Beats C#, and Python Beats HTML.
// HTML - 0 points
// C# - 3 points
// Python - 6 points
```

```
1 HOME_TEAM_WON = 1
2 def tournamentWinner(competitions, results):
3     # Write your code here.
4     currentBestTeam = ""
5     scores = {currentBestTeam:0}
6     for idx, competition in enumerate(competitions):
7         result = results[idx]
8         homeTeam, awayTeam = competition
9
10        winningTeam = homeTeam if result==HOME_TEAM_WON else awayTeam
11
12        updateScores(winningTeam,3,scores)
13
14        if scores[winningTeam]> scores[currentBestTeam]:
15            currentBestTeam = winningTeam
16
17    return currentBestTeam
18 def updateScores(team,points,scores):
19     if team not in scores:
20         scores[team] = 0
21     scores[team]+=points
```

non Constructible Change

总结：首先对列表进行排序。然后不断地对前面数进行求和操作。如果发现当前数-前面数的求和>1。表示最小不可创建的数为前面的数求和+1.否则也把这个数纳入求和之中。

```
1 def nonConstructibleChange(coins):
2     # Write your code here.
3     change_sum = 0
4     coins.sort()
5     for i in range(len(coins)):
```

```

6         if coins[i] - change_sum > 1:
7             output = change_sum + 1
8             return output
9         change_sum += coins[i]
10    output = change_sum+1
11    return output

```

find Closest Value In Bst

总结：closest指向绝对值最小的node。如果发现 $\text{target} - \text{current_node.value} > \text{target} - \text{closest}$ 的绝对值。则更新 $\text{closest} = \text{current_node.value}$ 。

然后对所有的node 进行遍历，如果 $\text{target} > \text{current_node.value}$ 则 $\text{current_node} = \text{current.right}$, 否则为 current_node.left 。

Sample Input

```

tree = 10
      /  \
     5    15
    /  \  /  \
   2    5 13  22
  /      \
 1         14
target = 12

```

Sample Output

```
13
```

```

1 def findClosestValueInBst(tree, target):
2     # Write your code here.
3     mingap = 0
4     pointer = tree
5     closest = pointer.value
6     while pointer is not None:
7         if abs(target - closest) > abs(target - pointer.value):
8             closest = pointer.value
9         if target > pointer.value:
10            pointer = pointer.right
11        elif target < pointer.value:
12            pointer = pointer.left
13        else:
14            break

```



```

15
16     return closest
17
18 # This is the class of the input tree. Do not edit.
19 class BST:
20     def __init__(self, value):
21         self.value = value
22         self.left = None
23         self.right = None
24
25 #tree 结构
26 {
27     "tree": {
28         "nodes": [
29             {"id": "10", "left": "5", "right": "15", "value": 10},
30             {"id": "15", "left": "13", "right": "22", "value": 15},
31             {"id": "22", "left": null, "right": null, "value": 22},
32             {"id": "13", "left": null, "right": "14", "value": 13},
33             {"id": "14", "left": null, "right": null, "value": 14},
34             {"id": "5", "left": "2", "right": "5-2", "value": 5},
35             {"id": "5-2", "left": null, "right": null, "value": 5},
36             {"id": "2", "left": "1", "right": null, "value": 2},
37             {"id": "1", "left": null, "right": null, "value": 1}
38         ],
39         "root": "10"
40     },
41     "target": 12
42 }

```

branch Sums 分支求和

总结：要求每条支链的和。首先需要知道遍历的结束条件是遍历到了叶子节点。

即左节点和右节点都为None。此时需要往sums列表中添加当前更新的求和的值。直接返回。

Sample Input

```
tree =      1
           /  \
          2    3
         /  \  /  \
        4    5 6    7
       /  \ /
      8   9 10
```

Sample Output

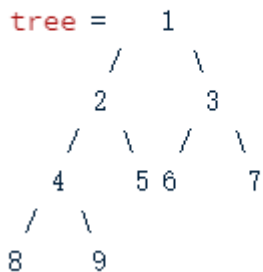
```
[15, 16, 18, 10, 11]
// 15 = 1 + 2 + 4 + 8
// 16 = 1 + 2 + 4 + 9
// 18 = 1 + 2 + 5 + 10
// 10 = 1 + 3 + 6
// 11 = 1 + 3 + 7
```

```
1 class BinaryTree:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7
8     def branchSums(root):
9         # Write your code here.
10        sums = []
11        calculateBranchSums(root, 0, sums)
12        return sums
13
14    def calculateBranchSums(node, runningSum, sums):
15        if node is None:
16            return
17
18        newRunningSum = runningSum + node.value
19        if node.left is None and node.right is None:
20            sums.append(newRunningSum)
21            return
22
23        calculateBranchSums(node.left, newRunningSum, sums)
24        calculateBranchSums(node.right, newRunningSum, sums)
```

node Depths

总结：重点还是要确定结束的条件，还是遍历到了叶子节点时。node 是none。否则，对应的 depth+1。

Sample Input



Sample Output

```

16
// The depth of the node with value 2 is 1.
// The depth of the node with value 3 is 1.
// The depth of the node with value 4 is 2.
// The depth of the node with value 5 is 2.
// Etc..
// Summing all of these depths yields 16.
  
```

```

1 def nodeDepths(root):
2     # Write your code here.
3     depth = 0
4     depth = cal_node_depths(root, depth)
5     return depth
6
7 def cal_node_depths(pointer, depth):
8     if pointer is None:
9         return 0
10    return depth + cal_node_depths(pointer.left, depth+1) \
11              + cal_node_depths(pointer.right, depth+1)
12
13 # This is the class of the input binary tree.
14 class BinaryTree:
15     def __init__(self, value):
16         self.value = value
17         self.left = None
  
```

```
18         self.right = None
19
```

depth First Search

总结：在构造类里面，self 代表构造的实体。深度优先遍历，是先序遍历，优先填充结果，然后不断重复。

```
1 class Node:
2     def __init__(self, name):
3         self.children = []
4         self.name = name
5
6     def addChild(self, name):
7         self.children.append(Node(name))
8         return self
9
10    def depthFirstSearch(self, array):
11        # Write your code here.
12        array.append(self.name)
13        for each in self.children:
14            each.depthFirstSearch(array)
15        return array
16
```

mini mum WaitingTime

总结：最小等待时间，就是应该从小到大的序列。

当前值得最小等待时间，就是到当前值为止，前面的等待时间。所以需要双循环。

```
1 def minimumWaitingTime(queries):
2     # Write your code here.
3     sum_delay = 0
4     queries.sort()
5     if queries is not None or len(queries)>1:
6         len_que = len(queries)
7         for i in range(len_que):
8             for j in range(0,i):
9                 sum_delay += queries[j]
10    return sum_delay
11
```

class Photos

总结：对列表进行从大到小进行排列。然后确定谁在前谁在后。

Sample Input

```
redShirtHeights = [5, 8, 1, 3, 4]
blueShirtHeights = [6, 9, 2, 4, 5]
```

Sample Output

```
true // Place all students with blue shirts in the back row.
```

```
1 def classPhotos(redShirtHeights, blueShirtHeights):
2     # Write your code here.
3     #we should sort the list,from the big to small
4     redShirtHeights.sort(reverse=True)
5     blueShirtHeights.sort(reverse=True)
6     flag = True
7     #we should find which should be in the front
8     Shirt_front = "red" if redShirtHeights[0]<blueShirtHeights[0] else "blue"
9
10    #to confirm the every num in front list should smaller than the behind
11
12    for num in range(len(redShirtHeights)):
13        #if the front is red,if the red is bigger/equal than blue return flase
14        if Shirt_front == "red":
15            if redShirtHeights[num]>= blueShirtHeights[num]:
16                flag = False
17        if Shirt_front == "blue":
18            if blueShirtHeights[num]>= redShirtHeights[num]:
19                flag = False
20
21    return flag
```

tandem Bicycle

总结：如果fastest = true 表示返回最大同步速度，如果=false 返回最小同步速度。最大同步速度，就是一个最大的配另一个最小的。最小同步速度就是两者最大配最大，最小配最小。

只要对排列的方式进行分类，此题就很好解了。

```
1 def tandemBicycle(redShirtSpeeds, blueShirtSpeeds, fastest):
```

```
2
```

```

3     #if fasetest = true ,we should sorted the list,one from the big to small,another is f
4     if fastest == True:
5         redShirtSpeeds.sort()
6         blueShirtSpeeds.sort(reverse = True)
7     #if false,we should sorted the list from small to big
8     else:
9         redShirtSpeeds.sort()
10        blueShirtSpeeds.sort()
11    #to compare the every num of the list to acquire the max num.
12    sum_output = 0
13    for num in range(len(redShirtSpeeds)):
14        sum_output+= redShirtSpeeds[num] if redShirtSpeeds[num]>blueShirtSpeeds[num]
15
16    return sum_output

```

remove Duplicates From LinkedList

Sample Input

```
linkedList = 1 -> 1 -> 3 -> 4 -> 4 -> 4 -> 5 -> 6 -> 6 // the head node with value 1
```

Sample Output

```
1 -> 3 -> 4 -> 5 -> 6 // the head node with value 1
```

总结：通过一个pointer 指针和一个指向其下一位的nextpointer指针。结束条件为None。
 当发现nextpointer.value == pointer.value时，变动nextpointer 即可。当发现不一样时则改变
 pointer.next = nextpointer pointer = nextpointer 改变pointer

```

1 class LinkedList:
2     def __init__(self, value):
3         self.value = value
4         self.next = None
5
6
7 def removeDuplicatesFromLinkedList(linkedList):
8     # Write your code here.
9     pointer = linkedList
10    while pointer is not None:
11        nextpointer = pointer.next
12        while nextpointer is not None and pointer.value == nextpointer.value:
13            nextpointer = nextpointer.next

```

```

14
15         pointer.next = nextpointer
16         pointer = nextpointer
17
18     return linkedList

```

get Nth Fib

总结：斐不那契数列，设置好收束的条件，通过递归调用即可。

```

1 def getNthFib(n):
2     # Write your code here.
3     if n == 1:
4         return 0
5     elif n == 2:
6         return 1
7     else:
8         return getNthFib(n-1)+getNthFib(n-2)

```

product Sum

Sample Input

```
array = [5, 2, [7, -1], 3, [6, [-13, 8], 4]]
```

Sample Output

```
12 // calculated as: 5 + 2 + 2 * (7 - 1) + 3 + 2 * (6 + 3 * (-13 + 8) + 4)
```

总结：该题总结起来很简单，当在列表中发现另一个列表，直接返回该列表里面所有数的求和，和列表深度的乘积。这样递归的条件就设置好了。

```

1 # Tip: You can use the type(element) function to check whether an item
2 # is a list or an integer.
3 def productSum(array,multiplier = 1):
4     sum_array = 0
5     for element in array:
6         if type(element) is list:
7             sum_array += productSum(element,multiplier+1)
8         else:
9             sum_array += element
10    return sum_array * multiplier

```

Binary Search

Sample Input

```
array = [0, 1, 21, 33, 45, 45, 61, 71, 72, 73]
target = 33
```

Sample Output

```
3
```

总结：二分法查找。

```
1 def binarySearch(array, target):
2     # Write your code here.
3     left = 0
4     right = len(array)-1
5     mid = (left + right)//2
6
7     while left <= right:
8         if array[mid]<target:
9             left = mid +1
10            mid =(left + right)//2
11        elif array[mid] > target:
12            right = mid -1
13            mid =(left + right)//2
14        elif array[mid] ==target:
15            return mid
16    else:
17        return -1
```

Find Three Largest Numbers

Sample Input

```
array = [141, 1, 17, -7, -17, -27, 18, 541, 8, 7, 7]
```

Sample Output

```
[18, 141, 541]
```

总结：不断地找到最大的那个数，将其放入一个列表中，并将最大的数remove掉就行了。

```
1 def findThreeLargestNumbers(array):
2     # Write your code here.
3     the_num_of_max = 3
4     the_output = []
```



```

5     for i in range(the_num_of_max):
6         max_num = max(array)
7         the_output.append(max_num)
8         array.remove(max_num)
9
10    the_output.sort()
11    return the_output

```

Bubble Sort

Sample Input

```
array = [8, 5, 2, 9, 5, 6, 3]
```

Sample Output

```
[2, 3, 5, 5, 6, 8, 9]
```

总结：冒泡排序，不断地找到最大的数，将其送到最后面，同时循环的返回从最后面逐步往前缩小。

```

1  def bubbleSort(array):
2      # Write your code here.
3      num_sorted = 0
4      while num_sorted < len(array):
5          #the example is means they don't into the below recycle.
6          for i in range(0, len(array)-1-num_sorted):
7              if array[i] > array[i+1]:
8                  swap(i, i+1, array)
9              num_sorted += 1
10     return array
11
12 def swap(i, j, array):
13     array[i], array[j] = array[j], array[i]

```

Insertion Sort

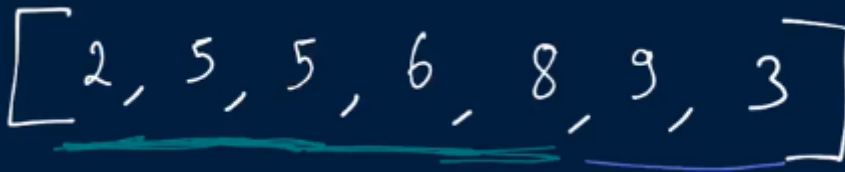
Sample Input

```
array = [8, 5, 2, 9, 5, 6, 3]
```

Sample Output

```
[2, 3, 5, 5, 6, 8, 9]
```

总结：将数组list分成2个部分，一个是前面已经排好序的绿色划线部分，一个是后面需要插入排序的部分，通过和最后一个绿色的数进行比较，不断进行交换，将数移动到合适的位置。



[2, 5, 5, 6, 8, 9, 3]

```
1 def insertionSort(array):
2     # Write your code here.
3     for unsorted_id in range(1, len(array)):
4         j = unsorted_id
5         while j > 0 and array[j] < array[j-1]:
6             swap(j, j-1, array)
7             j -= 1
8     return array
9
10
11 def swap(i, j, array):
12     array[i], array[j] = array[j], array[i]
```

Selection Sort

Sample Input

```
array = [8, 5, 2, 9, 5, 6, 3]
```

Sample Output

```
[2, 3, 5, 5, 6, 8, 9]
```

总结：将list分成2个部分，前面一个是已排序的部分，后面部分选出最小的一个数，将其移动到前面已排序好的部分，然后已排序的部分逐步往后退。

```
1 def selectionSort(array):
2     sorted_id = 0
3     while sorted_id < len(array)-1:
4         smallest_id = sorted_id
5         for i in range(sorted_id+1,len(array)):
6             if array[smallest_id] > array[i]:
7                 smallest_id = i
8         swap(sorted_id,smallest_id,array)
9         sorted_id+=1
10    return array
11
12 def swap(i,j,array):
13    array[i],array[j] = array[j],array[i]
```

Is Palindrome

Sample Input

```
string = "abcdcba"
```

Sample Output

```
true // it's written the same forward and backward
```

总结：通过前后两个指针来解决。

```
1 def isPalindrome(string):
2     begin_str = 0
3     end_str = len(string)-1
4     output_flag = True
5     while end_str > begin_str:
6         if string[begin_str] != string[end_str]:
7             output_flag = False
8         end_str -= 1
9         begin_str +=1
10
11    return output_flag
```

caesar Cipher Encryptor

Sample Input

```
string = "xyz"  
key = 2
```

Sample Output

```
"zab"
```

总结：这题并不难。首先要将string类型的字符，取出来将其转化为对应的ASCII码值。然后再加2. 这里又需要注意的一点，就是到了第26个字符之后要重头开始。所以需要%26。当然这一切都是基于 'a' 这个字符的基础之上的。

```
1 def caesarCipherEncryptor(string, key):  
2     # Write your code here.  
3     list_str = list(string)  
4     output_str = ''  
5     for i in range(len(string)):  
6         num_char = ord(list_str[i])+key  
7         if num_char > ord('z'):  
8             num_char = (num_char - ord('a'))%26 + ord('a')  
9             #output_str +=chr(num_char)  
10            list_str[i] = chr(num_char)  
11    #重新拼接成字符串。  
12    output_str ="".join(list_str)  
13    return output_str
```

run Length Encoding

Sample Input

```
string = "AAAAAAAAAAAAABBBCCDD"
```

Sample Output

```
"9A4A2B4C2D"
```

总结：2个条件，如果发现前一个数和后一个数不一样了，则转化为缩写。第二个条件，则是发现前一个数重复的次数达到9次，则强制进行转换。只要把预设的条件搞好，这道题目就很好解了。

```

1 def runLengthEncoding(string):
2     # Write your code here.
3     output_list = []
4     count = 1
5     for i in range(1, len(string)):
6         pre_char = string[i-1]
7         cur_char = string[i]
8         if pre_char != cur_char or count == 9:
9             output_list.append(str(count))
10            output_list.append(pre_char)
11            count = 0
12            count += 1
13
14    #deal the last run
15    output_list.append(str(count))
16    output_list.append(string[len(string)-1])
17
18    return "".join(output_list)

```

generate Document

Sample Input

```

characters = "Bstelhetsi ogEAxpelrt x "
document = "AlgoExpert is the Best!"

```

Sample Output

```
true
```

总结：比较document中的数可以从characters中生成。个人觉得最好的方法还是通过建立字典。如果在循环document 中发现有等于0的存在则报错。

```

1 def generateDocument(characters, document):
2     characters_dict = {}
3     document_flage = True
4     for character in characters:
5         if character not in characters_dict:
6             characters_dict[character] = 1
7         else:
8             characters_dict[character] += 1

```

```

9
10 for documnet_member in document:
11     if documnet_member not in characters_dict:
12         document_flage = False
13         break
14         #因为需要-1所以等于0就不行了
15     elif characters_dict[documnet_member] == 0:
16         document_flage = False
17         break
18     else:
19         characters_dict[documnet_member] -= 1
20
21 return document_flage

```

first Non Repeating Character

Sample Input

```
string = "abcdcaf"
```

Sample Output

```
1 // The first non-repeating character is "b" and is found at index 1.
```

总结：找到不重复的数，可以通过建立字典，找到第一个等于1的数就行了。

```

1 def firstNonRepeatingCharacter(string):
2     # Write your code here.
3     type_dict = {}
4     non_repeat = '-1'
5     for i in range(len(string)):
6         if string[i] not in type_dict:
7             type_dict[string[i]] = 1
8         else:
9             type_dict[string[i]] += 1
10
11     for each_key in type_dict.keys():
12         if type_dict[each_key] == 1:
13             non_repeat = each_key
14             break

```

```
15
16     if non_repeat == '-1':
17         return -1
18     else:
19         return string.find(non_repeat)
```

three Number Sum

Sample Input

```
array = [12, 3, 1, 2, -6, 5, -8, 6]
targetSum = 0
```

Sample Output

```
[[-8, 2, 6], [-8, 3, 5], [-6, 1, 5]]
```

总结：三重循环就可以了，实在想减少一个循环 可以通过字典进行哈希对照就行了。不过要注意不能产生哈希碰撞，这里不做推荐。

```
1 def threeNumberSum(array, targetSum):
2     # Write your code here.
3     output_list = []
4     #for begin to sorted
5     array.sort()
6     for first_num in range(len(array)):
7         for second_num in range(first_num+1, len(array)):
8             for third_num in range(second_num+1, len(array)):
9                 if array[first_num]+array[second_num]+array[third_num]==targetSum:
10                     output_list.append([array[first_num], array[second_num], array[third_num]])
11     return output_list
```

smallest Difference

Sample Input

```
arrayOne = [-1, 5, 10, 20, 28, 3]
arrayTwo = [26, 134, 135, 15, 17]
```

Sample Output

```
[28, 26]
```

总结：这题不用想太多，很简单。

```
1 def smallestDifference(arrayOne, arrayTwo):
2     minest_gap = abs(arrayOne[0] - arrayTwo[0])
3     minest_array1 = arrayOne[0]
4     minest_array2 = arrayTwo[0]
5
6     for i in range(0, len(arrayOne)):
7         for j in range(0, len(arrayTwo)):
8             if abs(arrayTwo[j] - arrayOne[i]) < minest_gap:
9                 minest_gap = abs(arrayTwo[j] - arrayOne[i])
10                minest_array1 = arrayOne[i]
11                minest_array2 = arrayTwo[j]
12
13
14     return [minest_array1, minest_array2]
```

move Element To End

总结：这题也没啥好说的。

Sample Input

```
array = [2, 1, 2, 2, 2, 3, 4, 2]
toMove = 2
```

Sample Output

```
[1, 3, 4, 2, 2, 2, 2, 2] // the numbers 1, 3, and 4 could be ordered differently
```

```
1 def moveElementToEnd(array, toMove):
2     begin_one = 0
```



```

3     end_one = len(array)-1
4
5     while begin_one < end_one:
6         if array[begin_one] != toMove:
7             begin_one +=1
8         else:
9             swap(begin_one,end_one,array)
10            end_one -=1
11
12     return array
13
14
15 def swap(i,j,array):
16     array[i],array[j] = array[j],array[i]

```

is Monotonic

总结：不是很难，检测到第一个，在去判断内部有没有反单调的存在即可。

Sample Input

```
array = [-1, -5, -10, -1100, -1100, -1101, -1102, -9001]
```

Sample Output

```
true
```

```

1 def isMonotonic(array):
2     isIncreasingFlag = None
3     #首先判断array是单调增函数还是单调减函数。
4     for i in range(0,len(array)-1):
5         if array[i] == array[i+1]:
6             continue
7         isIncreasingFlag = True if array[i] < array[i+1] else False
8         break
9
10    # 如果首先发现了他是单调增的函数，如果发现有存在array[i] >array[i+1] 则返回false
11    isMonotonicFlag = True
12    if isIncreasingFlag:

```

```

13         for i in range(0, len(array)-1):
14             if array[i] > array[i+1]:
15                 isMonotonicFlag = False
16                 break
17     elif isIncreasingFlag is False:
18         for i in range(0, len(array)-1):
19             if array[i+1] > array[i]:
20                 isMonotonicFlag = False
21                 break
22     else:
23         isMonotonicFlag = True
24     return isMonotonicFlag

```

spiral Traverse

Sample Input

```

array = [
    [1, 2, 3, 4],
    [12, 13, 14, 5],
    [11, 16, 15, 6],
    [10, 9, 8, 7],
]

```

Sample Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
```

总结：螺旋输出，按照顺时针，写好4个for循环即可，注意边界条件。

```

1 def spiralTraverse(array):
2     limit_long_upper = 0
3     limit_width_left = 0
4     limit_long_below = len(array)-1
5     limit_width_right = len(array[0])-1
6
7     output_list = []
8     #recycle the spiral
9     while limit_long_below >= limit_long_upper and limit_width_right >= limit_width_left:
10         for i in range(limit_width_left, limit_width_right+1):
11             output_list.append(array[limit_long_upper][i])
12

```

```

13         for i in range(limit_long_upper+1,limit_long_below+1):
14             output_list.append(array[i][limit_width_right])
15         #反向for循环用reversed即可
16         for i in reversed(range(limit_width_left,limit_width_right)):
17             #输出行，当限制的行相等时，表示已经循环过了
18             if limit_long_upper == limit_long_below:
19                 break
20             output_list.append(array[limit_long_below][i])
21
22         for i in reversed(range(limit_long_upper+1,limit_long_below)):
23             #输出列，当限制的列相等时，表示已经循环过了
24             if limit_width_left == limit_width_right:
25                 break
26             output_list.append(array[i][limit_width_left])
27
28
29         limit_long_upper +=1
30         limit_width_right -=1
31         limit_long_below -=1
32         limit_width_left +=1
33
34     return output_list

```

longest Peak (最长的峰顶)

For example, the integers `1, 4, 10, 2` form a peak, but the integers `4, 0, 10` don't and neither do the integers `1, 2, 2, 0`. Similarly, the integers `1, 2, 3` don't form a peak because there aren't any strictly decreasing integers after the `3`.

Sample Input

```
array = [1, 2, 3, 3, 4, 0, 10, 6, 5, -1, -3, 2, 3]
```

Sample Output

```
6 // 0, 10, 6, 5, -1, -3
```

总结：根据条件，分析得出peak的条件，即大于前一个数，同时也大于后一个数。当找到这个peak后，我们便根据这个点，往前，往后进行推导，判断这个peak的范围。

```

1 def longestPeak(array):
2     longest_count = 0
3     i = 1
4     top_peak = False
5     while i < len(array)-1:

```

```

6         #find the top of the peak
7         top_peak = array[i]>array[i-1] and array[i]> array[i+1]
8         if not top_peak:
9             i += 1
10            continue
11        begin_peak = i - 2
12        while begin_peak>=0 and array[begin_peak]<array[begin_peak+1]:
13            begin_peak +=1
14        end_peak = i + 2
15        while end_peak<=len(array)-1 and array[end_peak]<array[end_peak+1]:
16            end_peak += 1
17
18        current_count = end_peak - begin_peak -1
19        longest_count = max(longest_count,current_count)
20        i = end_peak
21
22    return longest_count

```

array Of Products (列的交叉乘积)

Sample Input

```
array = [5, 1, 4, 2]
```

Sample Output

```

[8, 40, 10, 20]
// 8 is equal to 1 x 4 x 2
// 40 is equal to 5 x 4 x 2
// 10 is equal to 5 x 1 x 2
// 20 is equal to 5 x 1 x 4

```

总结：没啥总结的，这题很简单。

```

1 def arrayOfProducts(array):
2     product_list = []
3     for i in range(len(array)):
4         product_result = 1
5         for j in range(len(array)):
6             if i != j:
7                 product_result *= array[j]
8         product_list.append(product_result)
9

```

```
10     return product_list
```

first Duplicate Value

Sample Input #1

```
array = [2, 1, 5, 2, 3, 3, 4]
```

Sample Output #1

```
2 // 2 is the first integer that appears more than once.  
// 3 also appears more than once, but the second 3 appears after the second 2.
```

Sample Input #2

```
array = [2, 1, 5, 3, 3, 2, 4]
```

Sample Output #2

```
3 // 3 is the first integer that appears more than once.  
// 2 also appears more than once, but the second 2 appears after the second 3.
```

总结：没啥总结的，这题很简单。

```
1 def firstDuplicateValue(array):  
2     new_list = []  
3     output = -1  
4     for element in array:  
5         if element not in new_list:  
6             new_list.append(element)  
7         else:  
8             output = element  
9             break  
10    return output
```

merge Overlapping Intervals

Sample Input

```
intervals = [[1, 2], [3, 5], [4, 7], [6, 8], [9, 10]]
```

Sample Output

```
[[1, 2], [3, 8], [9, 10]]  
// Merge the intervals [3, 5], [4, 7], and [6, 8].  
// The intervals could be ordered differently.
```

总结：确定好右边界的变动条件即可。还有排序方法可以用key= lambda x:x[0]简化。

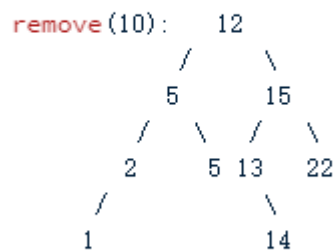
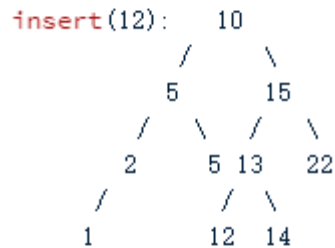
```
1 def mergeOverlappingIntervals(intervals):
2     # Sort the intervals
3     for i in range(len(intervals)):
4         smallest_id = i
5         for j in range(i+1,len(intervals)):
6             if intervals[j][0]<intervals[smallest_id][0]:
7                 smallest_id = j
8         swap(i,smallest_id,intervals)
9     #上面相当于这个
10    #sortedIntervals = sorted(intervals,key = lambda x:x[0])
11
12    left_interval = intervals[0][0]
13    right_interval = intervals[0][1]
14    interval_list = []
15    i = 1
16    for i in range(len(intervals)):
17        #如果当前的右边界大于后一个的左边，同时也大于后一个的右边，则写入的右边界不变
18        if intervals[i][0] <= right_interval and right_interval>intervals[i][1]:
19            continue
20        #右边界的变动条件
21        if intervals[i][0] <=right_interval<=intervals[i][1]:
22            right_interval = intervals[i][1]
23        #表示右边界小于左边界。有隔断
24        else:
25            interval_list.append([left_interval,right_interval])
26            left_interval = intervals[i][0]
27            right_interval = intervals[i][1]
28
29    #收尾工作
30    interval_list.append([left_interval,right_interval])
31    return interval_list
32
33 def swap(i,j,array):
34     array[i],array[j] = array[j],array[i]
```

BST Construction 二叉树的构建

```
// Assume the following BST has already been created:
```



```
// All operations below are performed sequentially.
```



contains(15): true

总结：1、首先是二叉树的插入，因为不存在无法插入的情况，所以通过一个True来保证不断地遍历和循环，直到找到满足条件的节点为止。当当前值大于目标值，且没有左节点，则插入，否则将当前值遍历到其左节点。右节点，是当前值比目标值小的情况，同理。

2、contains的大致思路和上面的insert类似。

3、remove时，最关键的一点是要时刻知道其的parentNode。如果找到了需要删除的节点，则进一步判断，如果当前节点的左右子节点都存在，则当前位置的值就等于该节点的右节点的最小值【右子树的最左边】，右子树移除这个节点【重复这个循环】。然后对parentNode是空的特殊情况进行判断，【表明一开始就找到了需要移除的值为root】。如果左子树存在，先将currentNode 转变为左子节点。最终还是需要右子节点，以右为准【找到右边的最小值作为根节点】。然后便是替换了，通过parentNode来替换掉currentNode

```
1 # Do not edit the class below except for
2 # the insert, contains, and remove methods.
3 # Feel free to add new properties and methods
4 # to the class.
5 class BST:
6     def __init__(self, value):
7         self.value = value
8         self.left = None
9         self.right = None
```

```

10
11 def insert(self, value):
12     # Write your code here.
13     # Do not edit the return statement of this method.
14     currentNode = self
15     while True:
16         if value < currentNode.value:
17             if currentNode.left is None:
18                 currentNode.left = BST(value)
19                 break
20             else:
21                 currentNode = currentNode.left
22         else:
23             if currentNode.right is None:
24                 currentNode.right = BST(value)
25                 break
26             else:
27                 currentNode = currentNode.right
28     return self
29
30 def contains(self, value):
31     # Write your code here.
32     currentNode = self
33     while currentNode is not None:
34         if value < currentNode.value:
35             currentNode = currentNode.left
36         elif value > currentNode.value:
37             currentNode = currentNode.right
38         else:
39             return True
40     return False
41
42 def remove(self, value, parentNode = None):
43     # Write your code here.
44     # Do not edit the return statement of this method.
45     currentNode = self
46
47     while currentNode is not None:

```

```

48         if value < currentNode.value:

```



```

49         parentNode = currentNode
50         currentNode = currentNode.left
51     elif value > currentNode.value:
52         parentNode = currentNode
53         currentNode = currentNode.right
54     #当前的值等于要找的值时
55     else:
56         if currentNode.left is not None and currentNode.right is not None:
57             #层层嵌套遍历
58             currentNode.value = currentNode.right.getMinValue()
59             currentNode.right.remove(currentNode.value, currentNode)
60         #deal with the root
61         elif parentNode is None:
62             if currentNode.left is not None:
63                 currentNode.value = currentNode.left.value
64                 currentNode.right = currentNode.left.right
65                 currentNode.left = currentNode.left.left
66             elif currentNode.right is not None:
67                 currentNode.value = currentNode.right.value
68                 currentNode.right = currentNode.right.right
69                 currentNode.left = currentNode.right.left
70             else:
71                 pass
72         #删除掉currentNode
73         elif parentNode.left == currentNode:
74             parentNode.left = currentNode.left if currentNode.left is not None else None
75         elif parentNode.right == currentNode:
76             parentNode.right = currentNode.right if currentNode.right is not None else None
77         break
78     return self
79
80 def getMinValue(self):
81     currentNode = self
82     while currentNode.left is not None:
83         currentNode = currentNode.left
84     return currentNode.value

```

validate Bst

Sample Input

```
tree = 10
      /  \
     5    15
    /  \  /  \
   2   5 13  22
  /       \
 1         14
```

Sample Output

```
true
```

总结：根据根节点的值进行遍历来确定，左边的值都比其小，右边的值都比其大。

```
1 # This is an input class. Do not edit.
2 class BST:
3     def __init__(self, value):
4         self.value = value
5         self.left = None
6         self.right = None
7
8
9 def validateBst(tree,min_value=float("-inf"),max_value=float('inf')):
10     if tree is None:
11         return True
12     if tree.value < min_value or tree.value >= max_value:
13         return False
14     valid_flag_left = validateBst(tree.left,min_value,tree.value)
15     valid_flag_right = validateBst(tree.right,tree.value,max_value)
16     return valid_flag_left and valid_flag_right
```

二叉树的遍历【三种遍历的方法】

总结：重点就是调节append的位置

```
1 def inOrderTraverse(tree, array):
2     if tree is not None:
3         inOrderTraverse(tree.left, array)
4         array.append(tree.value)
5         inOrderTraverse(tree.right, array)
```

```

6     return array
7
8
9 def preOrderTraverse(tree, array):
10     if tree is not None:
11         array.append(tree.value)
12         preOrderTraverse(tree.left, array)
13         preOrderTraverse(tree.right, array)
14     return array
15
16
17 def postOrderTraverse(tree, array):
18     if tree is not None:
19         postOrderTraverse(tree.left, array)
20         postOrderTraverse(tree.right, array)
21         array.append(tree.value)
22     return array

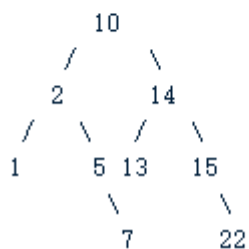
```

minHeight Bst 【通过一个序列构建一个最小高度二叉树】

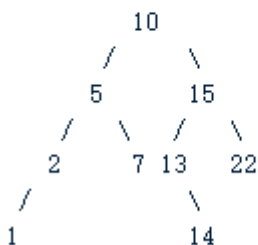
Sample Input

```
array = [1, 2, 5, 7, 10, 13, 14, 15, 22]
```

Sample Output



// This is one example of a BST with min height
 // that you could create from the input array.
 // You could create other BSTs with min height
 // from the same array; for example:



总结：通过二分法 不断地找到左子树和右子树的中间节点。来插入即可。

```

1 def minHeightBst(array):
2     array.sort()
3     return construct_bst(array, None, 0, len(array)-1)
4
5 def construct_bst(array, bst, left, right):
6     if right < left:
7         return
8     #root 节点 二分法来不断找寻中间节点
9     mid = (left+right)//2
10    if bst is None:
11        bst = BST(array[mid])
12    else:
13        bst.insert(array[mid])
14    construct_bst(array, bst, left, mid-1)
15    construct_bst(array, bst, mid+1, right)
16    return bst
17
18 class BST:
19     def __init__(self, value):
20         self.value = value
21         self.left = None
22         self.right = None
23
24     def insert(self, value):
25         if value < self.value:
26             if self.left is None:
27                 self.left = BST(value)
28             else:
29                 self.left.insert(value)
30         else:
31             if self.right is None:
32                 self.right = BST(value)
33             else:
34                 self.right.insert(value)

```

find KthLargestValue In Bst 【找到树里第K大的值】

Sample Input

```
tree =    15
        /  \
       5    20
      /  \  /  \
     2   5 17  22
    /  \
   1    3
k = 3
```

Sample Output

17

总结：这题比较简单，首先遍历所有的二叉树放入一个列表里，然后排序找到第k-1的数即可。

This is an input class. Do not edit.

```
1 class BST:
2     def __init__(self, value, left=None, right=None):
3         self.value = value
4         self.left = left
5         self.right = right
6
7
8     def findKthLargestValueInBst(tree, k):
9         #ergodic all the node and put node in a list
10        array_list = []
11        ergodic_all_node(tree,array_list)
12        array_list.sort(reverse=True)
13        print(array_list)
14        return array_list[k-1]
15
16
17    def ergodic_all_node(tree,array):
18        if tree is None:
19            return
20        array.append(tree.value)
21        ergodic_all_node(tree.left,array)
22        ergodic_all_node(tree.right,array)
```

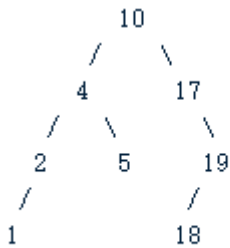
reconstruct Bst【根据先序遍历构建二叉树】

总结：先序遍历的第一个值永远是root节点，然后不断遍历，后一个作为其左节点，第一个大于第一个值的值作为右节点，不断循环遍历。

Sample Input

```
preOrderTraversalValues = [10, 4, 2, 1, 5, 17, 19, 18]
```

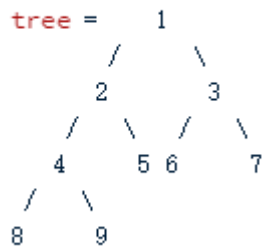
Sample Output



```
1 # This is an input class. Do not edit.
2 class BST:
3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7
8
9 def reconstructBst(preOrderTraversalValues):
10     if len(preOrderTraversalValues)==0:
11         return None
12
13     current_root_value = preOrderTraversalValues[0]
14     subtree_right_id = len(preOrderTraversalValues)
15
16     for id in range(1,subtree_right_id):
17         if preOrderTraversalValues[id] >= current_root_value:
18             subtree_right_id = id
19             break
20     subtree_left = reconstructBst(preOrderTraversalValues[1:subtree_right_id])
21     subtree_right = reconstructBst(preOrderTraversalValues[subtree_right_id:])
22     return BST(current_root_value,subtree_left,subtree_right)
23
```

invert Binary Tree

Sample Input



Sample Output

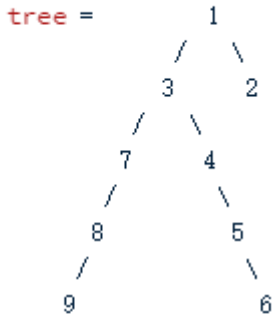


总结：用python解决这个问题相对比较简单，因为python中任何都是指针，所以直接交换就可以了。
将tree的左子树和右子树互相交换

```
1 def invertBinaryTree(tree):
2     if tree is None:
3         return
4     swap_tree(tree)
5     invertBinaryTree(tree.left)
6     invertBinaryTree(tree.right)
7
8     return tree
9
10 def swap_tree(tree):
11     tree.left,tree.right = tree.right,tree.left
12
13 # This is the class of the input binary tree.
14 class BinaryTree:
15     def __init__(self, value):
16         self.value = value
17         self.left = None
18         self.right = None
19
```

binary Tree Diameter

Sample Input



Sample Output

```
6 // 9 -> 8 -> 7 -> 3 -> 4 -> 5 -> 6
// There are 6 edges between the
// first node and the last node
// of this tree's longest path.
```

总结：这个题目的关键点是要理清直径和高度之间的关系。当前节点的直径是等于其左右节点高度的相加。同时通过maxDiameter来记录遍历过程中最大的diameter。

该题是需要用到遍历的，遍历的终止条件是找到叶子节点，然后由下往上进行遍历。如果该节点为None直接返回diameter为0，height为0.并通过longest_path_to_begin来统计currentNode的两个子树的高度之和和currentNode的diameter进行比较。最后并更新currentNode的diameter和height，其中height 需要在左右子树的最大值基础上+1。

其中递归的技巧需看代码体会。

```
1 # This is an input class. Do not edit.
2 class BinaryTree:
3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7 class Tree_diameter_height:
8     def __init__(self,diameter,height):
9         self.diameter = diameter
10        self.height = height
11
12
13 def binaryTreeDiameter(tree):
14     return get_tree_message(tree).diameter
15
16 def get_tree_message(tree):
```



```

17     if tree is None:
18         return Tree_diameter_height(0,0)
19     #从这里就开始递归了
20     left_tree_message = get_tree_message(tree.left)
21     right_tree_message = get_tree_message(tree.right)
22
23     longest_path_to_begin = left_tree_message.height + right_tree_message.height
24     max_diameter = max(left_tree_message.diameter, right_tree_message.diameter)
25     #当前路径，为子节点的最大路径和当前（左右子树高度之和）的最大值
26     current_diameter = max(max_diameter, longest_path_to_begin)
27     #当前高度= 左子树高度和右子树高度的最大值+1
28     current_height = 1+ max(left_tree_message.height, right_tree_message.height)
29
30     return Tree_diameter_height(current_diameter, current_height)

```

find Successor

Sample Input

```

tree =
    1
   / \
  2   3
 / \
4   5
/
6
node = 5

```

Sample Output

```

1
// This tree's in-order traversal order is:
// 6 -> 4 -> 2 -> 5 -> 1 -> 3
// 1 comes immediately after 5.

```

总结：这题相对就比较简单了，先通过中序遍历，将遍历的结果存储在list中，然后通过对list的判断，得到后面的遍历结果。

```

1 # This is an input class. Do not edit.
2 class BinaryTree:
3     def __init__(self, value, left=None, right=None, parent=None):
4         self.value = value

```

```
5         self.left = left
6         self.right = right
7         self.parent = parent
8
9
10 def InOrderTraverse1(tree,array):
11     if tree is None:
12         return
13     InOrderTraverse1(tree.left,array)
14     array.append(tree)
15     InOrderTraverse1(tree.right,array)
16
17     return array
18
19 def findSuccessor(tree, node):
20     list_array = []
21     list_array = InOrderTraverse1(tree,list_array)
22     for i,currentNode in enumerate(list_array):
23         if currentNode != node:
24             continue
25         if i == len(list_array)-1:
26             return None
27         return list_array[i+1]
```

height Balanced Binary Tree

Height Balanced Binary Tree 🟢 ☆

You're given the root node of a Binary Tree. Write a function that returns `true` if this Binary Tree is height balanced and `false` if it isn't.

A Binary Tree is height balanced if for each node in the tree, the difference between the height of its left subtree and the height of its right subtree is at most `1`.

Each `BinaryTree` node has an integer `value`, a `left` child node, and a `right` child node. Children nodes can either be `BinaryTree` nodes themselves or `None` / `null`.

Sample Input



Sample Output

true

总结：这题基本是**binary Tree Diameter**的引申，基本的解题思路类似，只不过一个是diameter，另一个是否平衡，是一个布尔变量。具体看代码体会一下就明白了。

```
1 # This is an input class. Do not edit.
2 class BinaryTree:
3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7 class Tree_Info:
8     def __init__(self, isBalanced, height):
9         self.isBalanced = isBalanced
10        self.height = height
11
12
13 def heightBalancedBinaryTree(tree):
14     # Write your code here.
15     tree_info = get_tree_info(tree)
16     return tree_info.isBalanced
17
18 def get_tree_info(node):
19     difference_most = 1
20
21     if node is None:
```

```

21         return Tree_Info(True, -1)
22
23     left_subtree_info = get_tree_info(node.left)
24     right_subtree_info = get_tree_info(node.right)
25
26     isBalanced = (left_subtree_info.isBalanced and
27                   right_subtree_info.isBalanced and
28                   abs(left_subtree_info.height - right_subtree_info.height) <= 1)
29
30     height = max(left_subtree_info.height, right_subtree_info.height) + 1
31     return Tree_Info(isBalanced, height)

```

max SubsetSum No Adjacent

Write a function that takes in an array of positive integers and returns the maximum sum of non-adjacent elements in the array.

If the input array is empty, the function should return `0`.

Sample Input

```
array = [75, 105, 120, 75, 90, 135]
```

Sample Output

```
330 // 75 + 120 + 135
```

总结：通过一个跟array相同长度的list，每位储存，与其前面不邻接元素的最大求和，这样遍历即可。同时去除掉特殊情况。

```

1 def maxSubsetSumNoAdjacent(array):
2     # Write your code here.
3     max_sum_result = 0
4     if len(array) == 0:
5         max_sum_result = 0
6     elif len(array) == 1:
7         max_sum_result = array[0]
8     elif len(array) == 2:
9         max_sum_result = max(array[0], array[1])
10    else:
11        max_sum = array[:]

```

```

12         max_sum[1] = max(array[0],array[1])
13         for i in range(2,len(array)):
14             max_sum[i] = max(max_sum[i-1],max_sum[i-2]+array[i])
15         max_sum_result = max_sum[-1]
16
17     return max_sum_result

```

number Of Ways To Make Change

Given an array of distinct positive integers representing coin denominations and a single non-negative integer `n` representing a target amount of money, write a function that returns the number of ways to make change for that target amount using the given coin denominations.

Note that an unlimited amount of coins is at your disposal.

Sample Input

```

n = 6
denoms = [1, 5]

```

Sample Output

```

2 // 1x1 + 1x5 and 6x1

```

总结：way表示，从自然数1开始到n之间的方法数目。然后计算target 中的每一个元素（从0到target）可以由denoms构成多少次。即可。

```

1 def numberOfWaysToMakeChange(n, denoms):
2     # Write your code here.
3     target = n
4     #initial ways
5     way = [0]*(target+1)
6     way[0] = 1
7
8     #target 中的每一个元素（从0到target）可以由denoms构成多少次。
9     denoms.sort()
10    for i in range(len(denoms)):
11        if denoms[i] <= target:
12            for j in range(denoms[i],target+1):
13                way[j] += way[j-denoms[i]]
14
15    return way[target]

```

min number Of Ways To Make Change

Min Number Of Coins For Change

Given an array of positive integers representing coin denominations and a single non-negative integer `n` representing a target amount of money, write a function that returns the smallest number of coins needed to make change for (to sum up to) that target amount using the given coin denominations.

Note that you have access to an unlimited amount of coins. In other words, if the denominations are `[1, 5, 10]`, you have access to an unlimited amount of `1` s, `5` s, and `10` s.

If it's impossible to make change for the target amount, return `-1`.

Sample Input

```
n = 7
denoms = [1, 5, 10]
```

Sample Output

```
3 // 2x1 + 1x5
```

总结：找到小于目标的最大数，将其置于倍数，逐步看是否大于目标。当大了，就逐步递归到下一个数。【思路一】

跟上面类似，不过这次不是相加，而是直接判断构成当前元素数目所需要的最小数目。

构成5 最少需要1个，构成6最少需要2个，构成7最少需要3个。

```
————— Test Case 1 —————
[0, inf, inf, inf, inf, inf, inf, inf]
[0, 1, 2, 3, 4, 5, 6, 7]
[0, 1, 2, 3, 4, 1, 2, 3]
[0, 1, 2, 3, 4, 1, 2, 3]
```

```
1 def minNumberOfCoinsForChange(n, denoms):
2     denoms_list = [float('inf') for i in range(n+1)]
3     denoms_list[0] = 0
4     print(denoms_list)
5     for denom in denoms:
6         for i in range(len(denoms_list)):
7             if denom <= i:
8                 denoms_list[i] = min(denoms_list[i], denoms_list[i-denom]+1)
```

```
9
10         print(denoms_list)
11     return denoms_list[n] if denoms_list[n] != float('inf') else -1
```

Levenshtein Distance

Difficulty:  Category:  Successful Submissions: 19,676+

Levenshtein Distance

Write a function that takes in two strings and returns the minimum number of edit operations that need to be performed on the first string to obtain the second string.

There are three edit operations: insertion of a character, deletion of a character, and substitution of a character for another.

Sample Input

```
str1 = "abc"
str2 = "yabd"
```

Sample Output

```
2 // insert "y"; substitute "c" for "d"
```

总结：如下图所示，构建第一行和第一列，然后根据字符串进行判断，相等则相当于忽略这个字符，回到 $r-1, c-1$ 的维度，如果不相等，则需要在相邻的维度进行判断，取最小值+1即可。最后返回右下角的值即可是变化的最小步骤了。

Levenshtein Distance

str1 = "abc"

str2 = "yabd"

按 Esc 即可退出全屏模式

		y	a	b	d
	0	1	2	3	4
a	1	1	1	2	3
b	2				
c	3				

if str1[r] == str2[c]: $E[r][c] = E[r-1][c-1]$

else: $E[r][c] = 1 + \min(E[r][c-1], E[r-1][c], E[r-1][c-1])$

```

1 def levenshteinDistance(str1, str2):
2     tablet = [[x for x in range(len(str2)+1)] for y in range(len(str1)+1)]
3     print(tablet)
4     for r in range(1, len(str1)+1):
5         tablet[r][0] = tablet[r-1][0] + 1
6     print(tablet)
7     for r in range(1, len(str1)+1):
8         for c in range(1, len(str2)+1):
9             if str2[c-1] == str1[r-1]:
10                 tablet[r][c] = tablet[r-1][c-1]
11             else:
12                 tablet[r][c] = 1 + min(tablet[r-1][c-1], tablet[r-1][c], tablet[r][c-1])
13     print(tablet)
14     return tablet[-1][-1]

```

number Of Ways To Traverse Graph

You're given two positive integers representing the width and height of a grid-shaped, rectangular graph. Write a function that returns the number of ways to reach the bottom right corner of the graph when starting at the top left corner. Each move you take must either go down or right. In other words, you can never move up or left in the graph.

For example, given the graph illustrated below, with `width = 2` and `height = 3`, there are three ways to reach the bottom right corner when starting at the top left corner:



1. Down, Down, Right
2. Right, Down, Down
3. Down, Right, Down

Note: you may assume that `width * height >= 2`. In other words, the graph will never be a 1x1 grid.

Sample Input

```
width = 4
height = 3
```

Sample Output

```
10
```

总结：可以参考递归的方式解决该问题，当距离右下角，横为1或者高度为1时，其的way就为1了。然后总的way就是，width-1 和height -1 的不断递归。

```
1 def numberOfWaysToTraverseGraph(width, height):
2     # Write your code here.
3     way = 0
4     way = solve_help(width,height,way)
5     return way
6
7 def solve_help(width,height,way):
8     if width == 1 or height ==1 :
9         way = 1
10    else:
```

```
11         way = solve_help(width-1,height,way)+solve_help(width,height-1,way)
12     return way
```

kadanes Algorithm

Write a function that takes in a non-empty array of integers and returns the maximum sum that can be obtained by summing up all of the integers in a non-empty subarray of the input array. A subarray must only contain adjacent numbers (numbers next to each other in the input array).

Sample Input

```
array = [3, 5, -9, 1, 3, -2, 3, 4, 7, 2, -9, 6, 3, 1, -5, 4]
```

Sample Output

```
19 // [1, 3, -2, 3, 4, 7, 2, -9, 6, 3, 1]
```

总结：通过一个max_end记录到**当前数前一位**（连续求和的最大值）。另一个sum_max记录到当前数前，连续求和的最大值。然后比较 max_end+当前数和当前数的大小，然后在再和sum_max进行比较。

```
1 def kadanesAlgorithm(array):
2     max_end = array[0]
3     sum_max = array[0]
4
5     for i in range(1,len(array)):
6         max_end = max(array[i],max_end+array[i])
7         sum_max = max(sum_max,max_end)
8     return sum_max
```

Single Cycle

You're given an array of integers where each integer represents a jump of its value in the array. For instance, the integer `2` represents a jump of two indices forward in the array; the integer `-3` represents a jump of three indices backward in the array.

If a jump spills past the array's bounds, it wraps over to the other side. For instance, a jump of `-1` at index `0` brings us to the last index in the array. Similarly, a jump of `1` at the last index in the array brings us to index `0`.

Write a function that returns a boolean representing whether the jumps in the array form a single cycle. A single cycle occurs if, starting at any index in the array and following the jumps, every element in the array is visited exactly once before landing back on the starting index.

Sample Input

```
array = [2, 3, 1, -4, -4, 2]
```

Sample Output

```
true
```

总结：题目的意思就是，从0开始按照数字遍历，可以遍历整个list的其他所有数。所以问题就好解决了。

```
1 def hasSingleCycle(array):
2     # Write your code here.
3     num_elements_visited = 0
4     current_id = 0
5     while num_elements_visited < len(array):
6         if num_elements_visited > 0 and current_id == 0:
7             return False
8         num_elements_visited += 1
9         current_id = get_next_id(current_id, array)
10    return current_id == 0
11
12 def get_next_id(current_id, array):
13     jump = array[current_id]
14     next_id = (current_id + jump) % len(array)
15     return next_id if next_id >= 0 else next_id + len(array)
```

breadth First Search (层次遍历)

You're given a `Node` class that has a `name` and an array of optional `children` nodes. When put together, nodes form an acyclic tree-like structure.

Implement the `breadthFirstSearch` method on the `Node` class, which takes in an empty array, traverses the tree using the Breadth-first Search approach (specifically navigating the tree from left to right), stores all of the nodes' names in the input array, and returns it.

If you're unfamiliar with Breadth-first Search, we recommend watching the Conceptual Overview section of this question's video explanation before starting to code.

Sample Input

```
graph = A
  / | \
 B  C  D
 / \ / \
E  F G  H
 / \ \
I  J  K
```

Sample Output

```
["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K"]
```

总结：取出一个数，不断地遍历其子节点即可。从第一位pop即可。

```
1 # Do not edit the class below except
2 # for the breadthFirstSearch method.
3 # Feel free to add new properties
4 # and methods to the class.
5 class Node:
6     def __init__(self, name):
7         self.children = []
8         self.name = name
9
10    def addChild(self, name):
11        self.children.append(Node(name))
12        return self
13
14    def breadthFirstSearch(self, array):
15        queue = [self]
16        while len(queue)>0:
17            current_node = queue.pop(0)
```

```

18         array.append(current_node.name)
19         for child in current_node.children:
20             queue.append(child)
21     return array

```

River Sizes

River Sizes 🟢 ☆

You're given a two-dimensional array (a matrix) of potentially unequal height and width containing only `0`s and `1`s. Each `0` represents land, and each `1` represents part of a river. A river consists of any number of `1`s that are either horizontally or vertically adjacent (but not diagonally adjacent). The number of adjacent `1`s forming a river determine its size.

Note that a river can twist. In other words, it doesn't have to be a straight vertical line or a straight horizontal line; it can be L-shaped, for example.

Write a function that returns an array of the sizes of all rivers represented in the input matrix. The sizes don't need to be in any particular order.

Sample Input

```

matrix = [
    [1, 0, 0, 1, 0],
    [1, 0, 1, 0, 0],
    [0, 0, 1, 0, 1],
    [1, 0, 1, 0, 1],
    [1, 0, 1, 1, 0],
]

```

Sample Output

```
[1, 2, 2, 2, 5] // The numbers could be ordered differently.
```

```

// The rivers can be clearly seen here:
// [
//   [1,  ,  , 1,  ],
//   [1,  , 1,  ,  ],
//   [ ,  , 1,  , 1],
//   [1,  , 1,  , 1],
//   [1,  , 1, 1,  ],
// ]

```

总结：首先需要对整个表进行遍历，同时设置一个visited列表，表示该节点有没有访问过。当检测到1的时候，首先我们需要将该river size的尺寸+1。同时遍历周围是否有没访问过的点，将没访问的节点

加入下步遍历的列表，进行循环。

```
1 def riverSizes(matrix):
2     sizes = []
3     visited = [[False for value in row] for row in matrix]
4     for i in range(len(matrix)):
5         for j in range(len(matrix[i])):
6             if visited[i][j]:
7                 continue
8             traverseNode(i,j,matrix,visited,sizes)
9     return sizes
10
11 def traverseNode(i,j,matrix,visited,sizes):
12     current_river_size = 0
13     nodes_to_explore = [[i,j]]
14     while len(nodes_to_explore):
15         current_node = nodes_to_explore.pop()
16         i = current_node[0]
17         j = current_node[1]
18         if visited [i][j]:
19             continue
20         visited[i][j] = True
21         if matrix[i][j] == 0:
22             continue
23         current_river_size +=1
24         unvisited_neighbors = get_unvisited_neighbors(i,j,matrix,visited)
25         for neighbor in unvisited_neighbors:
26             nodes_to_explore.append(neighbor)
27     if current_river_size > 0:
28         sizes.append(current_river_size)
29
30 def get_unvisited_neighbors(i,j,matrix,visited):
31     unvisited_neighbors = []
32     if i > 0 and not visited[i-1][j]:
33         unvisited_neighbors.append([i-1,j])
34     if i < len(matrix)-1 and not visited[i+1][j]:
35         unvisited_neighbors.append([i+1,j])
36     if j > 0 and not visited[i][j-1]:
37         unvisited_neighbors.append([i,j-1])
38     if j < len(matrix[0])-1 and not visited[i][j+1]:
```

```
39         unvisited_neighbors.append([i,j+1])
40     return unvisited_neighbors
```

Youngest Common Ancestor

Difficulty:  Category:  Successful Submissions: 19,491+

Youngest Common Ancestor

You're given three inputs, all of which are instances of an `AncestralTree` class that have an `ancestor` property pointing to their youngest ancestor. The first input is the top ancestor in an ancestral tree (i.e., the only instance that has no ancestor--its `ancestor` property points to `None` / `null`), and the other two inputs are descendants in the ancestral tree.

Write a function that returns the youngest common ancestor to the two descendants.

Note that a descendant is considered its own ancestor. So in the simple ancestral tree below, the youngest common ancestor to nodes A and B is node A.

```
// The youngest common ancestor to nodes A and B is node A.
  A
 /
B
```

```
// The nodes are from the ancestral tree below.
topAncestor = node A
descendantOne = node E
descendantTwo = node I
```



Sample Output

```
node B
```

总结：整体思路，首先找到descendant 中层级较低地那个节点。首先，将比较低地那个节点地层级逐步拔高，拔到和另一个节点相同层级地位置。如果这两个节点并不相等，则判断另一个节点并非为当前节点地ancestor。然后就是两个节点同时往上进行遍历，直到找到相同地节点为止。

```

1 # This is an input class. Do not edit.
2 class AncestralTree:
3     def __init__(self, name):
4         self.name = name
5         self.ancestor = None
6
7
8 def getYoungestCommonAncestor(topAncestor, descendantOne, descendantTwo):
9     # Write your code here.
10    depth_one = getDescendantDepth(descendantOne, topAncestor)
11    depth_two = getDescendantDepth(descendantTwo, topAncestor)
12    if depth_one > depth_two:
13        return backtrackAncestralTree(descendantOne, descendantTwo, depth_one - depth_two)
14    else:
15        return backtrackAncestralTree(descendantTwo, descendantOne, depth_two - depth_one)
16
17 def getDescendantDepth(descendant, topAncestor):
18     depth = 0
19     while descendant != topAncestor:
20         depth += 1
21         descendant = descendant.ancestor
22     return depth
23
24 def backtrackAncestralTree(lowerDescendant, higherDescendant, diff):
25     while diff > 0:
26         lowerDescendant = lowerDescendant.ancestor
27         diff -= 1
28     while lowerDescendant != higherDescendant:
29         lowerDescendant = lowerDescendant.ancestor
30         higherDescendant = higherDescendant.ancestor
31     return lowerDescendant

```

remove Islands

Sample Input

```
matrix =  
[  
  [1, 0, 0, 0, 0, 0],  
  [0, 1, 0, 1, 1, 1],  
  [0, 0, 1, 0, 1, 0],  
  [1, 1, 0, 0, 1, 0],  
  [1, 0, 1, 1, 0, 0],  
  [1, 0, 0, 0, 0, 1],  
]
```

Sample Output

```
[  
  [1, 0, 0, 0, 0, 0],  
  [0, 0, 0, 1, 1, 1],  
  [0, 0, 0, 0, 1, 0],  
  [1, 1, 0, 0, 1, 0],  
  [1, 0, 0, 0, 0, 0],  
  [1, 0, 0, 0, 0, 1],  
]
```

```
// The islands that were removed can be clearly seen here:  
// [  
//   [ , , , , , ],  
//   [ , 1, , , , ],  
//   [ , , 1, , , ],  
//   [ , , , , , ],  
//   [ , , 1, 1, , ],  
//   [ , , , , , ],  
// ]
```

总结：首先将与边界相连的 1【首先找到处于边界的1，然后通过边界的1，将与其相连的1 找出来。】找出来，然后根据ones_connect_to_border[row][col]来替换掉非border的island。

```
1 def removeIslands(matrix):  
2     ones_connect_to_border = [[False for col in matrix[0]] for row in matrix]  
3  
4     #find all the 1s that are not islands  
5     for row in range(len(matrix)):  
6         for col in range(len(matrix[row])):  
7             rowIsBorder = row == 0 or row == len(matrix)-1  
8             colIsBorder = col == 0 or col == len(matrix[row])-1  
9             isBorder = rowIsBorder or colIsBorder
```

```

10         if not isBorder:
11             continue
12
13         if matrix[row][col] == 0:
14             continue
15
16         findOnesConnectedToBorder(matrix, row, col, ones_connect_to_border)
17 #根据ones_connect_to_border[row][col]来替换掉非border的island
18 for row in range(1, len(matrix)-1):
19     for col in range(1, len(matrix[row])-1):
20         if ones_connect_to_border[row][col]:
21             continue
22         matrix[row][col] = 0
23
24     return matrix
25
26 def findOnesConnectedToBorder(matrix, startRow, startCol, ones_connect_to_border):
27     stack = [(startRow, startCol)]
28     while len(stack) > 0:
29         currentPosition = stack.pop()
30         currentRow, currentCol = currentPosition
31         #首先判断有没有访问
32         alreadyVisited = ones_connect_to_border[currentRow][currentCol]
33         if alreadyVisited:
34             continue
35         #访问后跳过，未访问置true
36         ones_connect_to_border[currentRow][currentCol] = True
37         #查找其对应的neighbors，并将为1的neighbor填充进stack中。
38         neighbors = getNeighbors(matrix, currentRow, currentCol)
39
40         for neighbor in neighbors:
41             row, col = neighbor
42
43             if matrix[row][col] != 1:
44                 continue
45             stack.append(neighbor)
46
47 def getNeighbors(matrix, row, col):
48     neighbors = []

```

```

49
50     numRows = len(matrix)
51     numCols = len(matrix[row])
52
53     if row - 1 >= 0:
54         neighbors.append((row-1,col))
55     if row + 1 < numRows:
56         neighbors.append((row+1,col))
57     if col + 1 < numCols:
58         neighbors.append((row,col+1))
59     if col - 1 >= 0:
60         neighbors.append((row,col-1))
61     return neighbors
62

```

cycle In Graph

Sample Input

```

edges = [
    [1, 3],
    [2, 3, 4],
    [0],
    [],
    [2, 5],
    [],
]

```

总结：通过一个visited表示访问过的节点，currentlyInStack表示，通过当前访问过的节点，通过边进行不断地遍历，将当前访问过地节点，存储在currentlyInStack中，如果重新从edges中读取到该节点，则表示形成了循环。如果没有，则表示当前遍历没有循环，则visited节点往前+1，同时将currentlyInStack[node]置为false。前面通过循环逐步从node 0 进行遍历。

Sample Output

```

true
// There are multiple cycles in this graph:
// 1) 0 -> 1 -> 2 -> 0
// 2) 0 -> 1 -> 4 -> 2 -> 0
// 3) 1 -> 2 -> 0 -> 1
// These are just 3 examples; there are more.

```

```

1 def cycleInGraph(edges):
2     numberOfNodes = len(edges)
3     visited = [False for _ in range(numberOfNodes)]
4     currentlyInStack = [False for _ in range(numberOfNodes)]
5
6     for node in range(numberOfNodes):
7         if visited[node]:
8             continue
9
10        containsCycle = isNodeInCycle(node, edges, visited, currentlyInStack)
11        if containsCycle:
12            return True
13    return False
14
15 def isNodeInCycle(node, edges, visited, currentlyInStack):
16     visited[node] = True
17     currentlyInStack[node] = True
18     neighbors = edges[node]
19
20     for neighbor in neighbors:
21         if not visited[neighbor]:
22             containsCycle = isNodeInCycle(neighbor, edges, visited, currentlyInStack)
23             if containsCycle:
24                 return True
25         #存在已访问过 但是未存储在stack的node (形成cycle)
26         elif currentlyInStack[neighbor]:
27             return True
28     currentlyInStack[node] = False
29     return False

```

minimum Passes Of Matrix (邻里反转)

题目意思：数列内部，当其中一个负数其neighborhood发现有一个正数，则其进行反转（变成正数），然后不断地重复该过程，知道数列中所有的负数都变成正数。如果无法全部变成正数，则返回-1，如果可以，则返回遍历整个数列的次数。

Sample Input

```
matrix = [  
    [0, -1, -3, 2, 0],  
    [1, -2, -5, -1, -3],  
    [3, 0, 0, -4, -1],  
]
```

Sample Output

3

个人在写的时候的思路：

- 1、首先我们得找到matrix 里面的正数。并把它对应的neighbors 记录下来。
- 2、每找到一个正数，就开始反转他的左邻右舍。【要判断他的左邻右舍小于0 才开始反转】，如果发现他的左邻右舍大于0，则把该邻居变为current_node。
- 3、只有发生翻转的变动，才计入变动因素+1。
- 4、老师给的通过两个链表动态地存储目前的正数的情况。并将翻转后的结果放入后一个链表之中。最后通过遍历查找是否还有负数来判断是否完成。

```
1 def minimumPassesOfMatrix(matrix):  
2     passes = convertNegatives(matrix)  
3     return passes - 1 if not containsNegative(matrix) else -1  
4  
5 def convertNegatives(matrix):  
6     #找到当前数列中为正数的点的位置，并整合成一个列表。  
7     nextPassQueue = getAllPositivePositions(matrix)  
8  
9     passes = 0  
10  
11     while len(nextPassQueue) > 0:  
12         currentPassQueue = nextPassQueue  
13         nextPassQueue = []  
14         while len(currentPassQueue) > 0:  
15             currentRow, currentCol = currentPassQueue.pop(0)  
16             #返回当前点的附近的邻里节点  
17             adjacentPositions = getAdjacentPositions(currentRow, currentCol, matrix)  
18             #当邻里节点出现小于0的情况，则乘以-1.  
19             for position in adjacentPositions:
```

```

20         row,col = position
21
22         value = matrix[row][col]
23         if value < 0:
24             matrix[row][col]*=-1
25             nextPassQueue.append([row,col])
26
27         passes += 1
28
29     return passes
30
31
32 def getAllPositivePositions(matrix):
33     positivePositions = []
34
35     for row in range(len(matrix)):
36         for col in range(len(matrix[row])):
37             value = matrix[row][col]
38             if value > 0 :
39                 positivePositions.append([row,col])
40
41     return positivePositions
42
43
44 def getAdjacentPositions(row,col,matrix):
45     adjacentPositions = []
46
47     if row > 0:
48         adjacentPositions.append([row-1,col])
49
50     if row < len(matrix)-1:
51         adjacentPositions.append([row+1,col])
52
53     if col > 0:
54         adjacentPositions.append([row,col-1])
55
56     if col < len(matrix[0])-1:
57         adjacentPositions.append([row,col+1])
58
59     return adjacentPositions
60
61
62 def containsNegative(matrix):
63     for row in matrix:
64         for value in row:
65             if value < 0:
66                 return True
67
68     return False

```

task Assignment (任务分配)

Task Assignment

You're given an integer `k` representing a number of workers and an array of positive integers representing durations of tasks that must be completed by the workers. Specifically, each worker must complete two unique tasks and can only work on one task at a time. The number of tasks will always be equal to `2k` such that each worker always has exactly two tasks to complete. All tasks are independent of one another and can be completed in any order. Workers will complete their assigned tasks in parallel, and the time taken to complete all tasks will be equal to the time taken to complete the longest pair of tasks (see the sample output for an explanation).

Sample Input

```
k = 3
tasks = [1, 3, 5, 3, 1, 4]
```

Sample Output

```
[
  [0, 2], // tasks[0] = 1, tasks[2] = 5 | 1 + 5 = 6
  [4, 5], // tasks[4] = 1, tasks[5] = 4 | 1 + 4 = 5
  [1, 3], // tasks[1] = 3, tasks[3] = 3 | 3 + 3 = 6
] // The fastest time to complete all tasks is 6.
```

// Note: there are multiple correct answers for this sample input.

思路：通过字典，储存当前任务完成时间对应其在列表中的位置信息。对任务进行排序，将前后一大一小进行配对组合起来即可。

```
1 def taskAssignment(k, tasks):
2     # Write your code here.
3     pairedTasks = []
4     taskDurationsToIndices = getTaskDurationsToIndices(tasks)
5
```

```

6     sortedTasks = sorted(tasks)
7     #对排序后的数列进行配对组合，并将配对完成的其字典中的数据-1
8     for idx in range(k):
9         task1Duration = sortedTasks[idx]
10        indicesWithTask1Duration = taskDurationsToIndices[task1Duration]
11        task1Index = indicesWithTask1Duration.pop()
12
13        task2SortedIndex = len(tasks) - 1 - idx
14        task2Duration = sortedTasks[task2SortedIndex]
15        indicesWithTask2Duration = taskDurationsToIndices[task2Duration]
16        task2Index = indicesWithTask2Duration.pop()
17
18        pairedTasks.append([task1Index, task2Index])
19
20    return pairedTasks
21    #构建任务位置信息的字典
22    def getTaskDurationsToIndices(tasks):
23        taskDurationsToIndices = {}
24
25        for idx, taskDuration in enumerate(tasks):
26            if taskDuration in taskDurationsToIndices:
27                taskDurationsToIndices[taskDuration].append(idx)
28            else:
29                taskDurationsToIndices[taskDuration] = [idx]
30    return taskDurationsToIndices

```

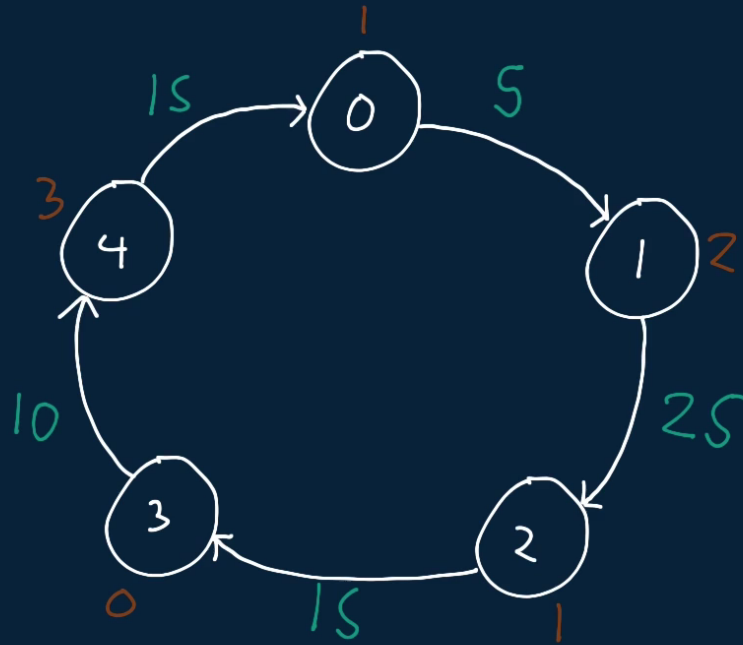
valid Starting City (适合起始的城市)

Valid Starting City

distances = [5, 25, 15, 10, 15]

fuel = [1, 2, 1, 0, 3]

mpg = 10



总结：这题的一个关键点就在于一定有且只有一个正确答案，比如这题总共行驶距离为70 = 10 * fuel。故从开始遍历这个数组，当发现油不够的时候，就从不够的地方重新开始即可。因为只要不够就无法完成循环，而该题一定有个正确答案，所以只要遍历一边后，发现没问题的点就一定能继续下去。所以根据这个思路，改题的解法就简单了。

```
1 def validStartingCity(distances, fuel, mpg):
2     numberOfCities = len(distances)
3     milesRemaining = 0
4
5     indexOfStartingCityCandidate = 0
6     milesRemainingAtStartingCityCandidate = 0
7
8     for cityIdx in range(1, numberOfCities):
9         distanceFromPreviousCity = distances[cityIdx - 1]
10        fuelFromPreviousCity = fuel[cityIdx - 1]
11        milesRemaining += fuelFromPreviousCity * mpg - distanceFromPreviousCity
12
13        if milesRemaining < milesRemainingAtStartingCityCandidate:
14            milesRemainingAtStartingCityCandidate = milesRemaining
15            indexOfStartingCityCandidate = cityIdx
16    return indexOfStartingCityCandidate
```

Min Heap Construction

Implement a `MinHeap` class that supports:

- Building a Min Heap from an input array of integers.
- Inserting integers in the heap.
- Removing the heap's minimum / root value.
- Peeking at the heap's minimum / root value.
- Sifting integers up and down the heap, which is to be used when inserting and removing values.

Note that the heap should be represented in the form of an array.

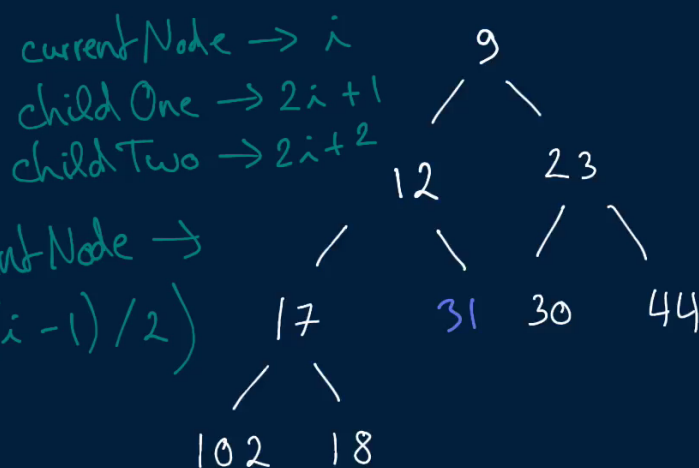
Sample Usage

```
array = [48, 12, 24, 7, 8, -5, 24, 391, 24, 56, 2, 6, 8, 41]

// All operations below are performed sequentially.
MinHeap(array): - // instantiate a MinHeap (calls the buildHeap method and populates the heap)
buildHeap(array): - [-5, 2, 6, 7, 8, 8, 24, 391, 24, 56, 12, 24, 48, 41]
insert(76): - [-5, 2, 6, 7, 8, 8, 24, 391, 24, 56, 12, 24, 48, 41, 76]
peek(): -5
remove(): -5 [2, 7, 6, 24, 8, 8, 24, 391, 76, 56, 12, 24, 48, 41]
peek(): 2
remove(): 2 [6, 7, 8, 24, 8, 24, 24, 391, 76, 56, 12, 41, 48]
peek(): 6
insert(87): - [6, 7, 8, 24, 8, 24, 24, 391, 76, 56, 12, 41, 48, 87]
```

Min Heap Construction

- Build Heap
- Sift Down
- Sift Up
- Insert
- Remove



0 1 2 3 4 5 6 7 8 1000

[9, 12, 23, 17, 31, 30, 44, 102, 18]

```

1 class MinHeap:
2     def __init__(self, array):
3         # Do not edit the line below.
4         self.heap = self.buildHeap(array)
5
6     def buildHeap(self, array):
7         firstParentIdx = (len(array)-2) // 2
8         for currentIdx in reversed(range(firstParentIdx + 1)):
9             self.siftDown(currentIdx, len(array)-1, array)
10        return array
11
12    def siftDown(self, currentIdx, endIdx, heap):
13        childOneIdx = currentIdx * 2 + 1
14        while childOneIdx <= endIdx:
15            childTwoIdx = currentIdx * 2 + 2 if currentIdx * 2 + 2 <= endIdx else -1
16            if childTwoIdx != -1 and heap[childTwoIdx] < heap[childOneIdx]:
17                idxToSwap = childTwoIdx
18            else:
19                idxToSwap = childOneIdx
20            if heap[idxToSwap] < heap[currentIdx]:
21                self.swap(currentIdx, idxToSwap, heap)
22                currentIdx = idxToSwap
23                childOneIdx = currentIdx * 2 + 1
24            else:
25                return
26
27    def siftUp(self, currentIdx, heap):
28        parentIdx = (currentIdx - 1) // 2
29        while currentIdx > 0 and heap[currentIdx] < heap[parentIdx]:
30            self.swap(currentIdx, parentIdx, heap)
31            currentIdx = parentIdx
32            parentIdx = (currentIdx - 1) // 2
33
34    def peek(self):
35        # Write your code here.
36        return self.heap[0]
37
38    def remove(self):

```

```

39         self.swap(0, len(self.heap)-1, self.heap)
40         valueToRemove = self.heap.pop()
41         self.siftDown(0, len(self.heap)-1, self.heap)
42         return valueToRemove
43
44     def insert(self, value):
45         # Write your code here.
46         self.heap.append(value)
47         self.siftUp(len(self.heap)-1, self.heap)
48     def swap(self, i, j, heap):
49         heap[i], heap[j] = heap[j], heap[i]

```

Doubly Linked List

Sample Usage

```

// Assume the following linked list has already been created:
1 <-> 2 <-> 3 <-> 4 <-> 5
// Assume that we also have the following stand-alone nodes:
3, 3, 6
setHead(4): 4 <-> 1 <-> 2 <-> 3 <-> 5 // set the existing node with value 4 as the head
setTail(6): 4 <-> 1 <-> 2 <-> 3 <-> 5 <-> 6 // set the stand-alone node with value 6 as
insertBefore(6, 3): 4 <-> 1 <-> 2 <-> 5 <-> 3 <-> 6 // move the existing node with value 6 to before 3
insertAfter(6, 3): 4 <-> 1 <-> 2 <-> 5 <-> 3 <-> 6 <-> 3 // insert a stand-alone node with value 3 after 6
insertAtPosition(1, 3): 3 <-> 4 <-> 1 <-> 2 <-> 5 <-> 3 <-> 6 <-> 3 // insert a stand-alone node with value 3 at position 1
removeNodesWithValue(3): 4 <-> 1 <-> 2 <-> 5 <-> 6 // remove all nodes with value 3
remove(2): 4 <-> 1 <-> 5 <-> 6 // remove the existing node with value 2
containsNodeWithValue(5): true

```

```

1 # This is an input class. Do not edit.
2 class Node:
3     def __init__(self, value):
4         self.value = value
5         self.prev = None
6         self.next = None
7
8
9 # Feel free to add new properties and methods to the class.
10 class DoublyLinkedList:
11     def __init__(self):

```

```
12         self.head = None
13         self.tail = None
14
15     def setHead(self, node):
16         # Write your code here.
17         if self.head is None:
18             self.head = node
19             self.tail = node
20             return
21         self.insertBefore(self.head, node)
22
23     def setTail(self, node):
24         # Write your code here.
25         if self.tail is None:
26             self.setHead(node)
27             return
28         self.insertAfter(self.tail, node)
29
30     def insertBefore(self, node, nodeToInsert):
31         if nodeToInsert == self.head and nodeToInsert == self.tail:
32             return
33         self.remove(nodeToInsert)
34         nodeToInsert.prev = node.prev
35         nodeToInsert.next = node
36         if node.prev is None:
37             self.head = nodeToInsert
38         else:
39             node.prev.next = nodeToInsert
40         node.prev = nodeToInsert
41
42     def insertAfter(self, node, nodeToInsert):
43         if nodeToInsert == self.head and nodeToInsert == self.tail:
44             return
45         self.remove(nodeToInsert)
46         nodeToInsert.prev = node
47         nodeToInsert.next = node.next
48         if node.next is None:
49             self.tail = nodeToInsert
50         else:
```

```
51         node.next.prev = nodeToInsert
52     node.next = nodeToInsert
53
54     def insertAtPosition(self, position, nodeToInsert):
55         if position == 1:
56             self.setHead(nodeToInsert)
57             return
58         node = self.head
59         currentPosition = 1
60         while node is not None and currentPosition != position:
61             node = node.next
62             currentPosition += 1
63         if node is not None:
64             self.insertBefore(node, nodeToInsert)
65         else:
66             self.setTail(nodeToInsert)
67
68     def removeNodesWithValue(self, value):
69         node = self.head
70         while node is not None:
71             nodeToRemove = node
72             node = node.next
73             if nodeToRemove.value == value:
74                 self.remove(nodeToRemove)
75
76     def remove(self, node):
77         # Write your code here.
78         if node == self.head:
79             self.head = self.head.next
80         if node == self.tail:
81             self.tail = self.tail.prev
82         self.removeNodeBindings(node)
83
84     def containsNodeWithValue(self, value):
85         node = self.head
86         while node is not None and (node.value != value):
87             node = node.next
88         return node is not None
89
```

```

90     def removeNodeBindings(self,node):
91         if node.prev is not None:
92             node.prev.next = node.next
93         if node.next is not None:
94             node.next.prev = node.prev
95         node.prev = None
96         node.next = None

```

Remove Kth Node From End

题意：一看就明白了，将距离链表尾部K的数移掉。

Sample Input

```

head = 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 // the head node with value 0
k = 4

```

Sample Output

```

// No output required.
// The 4th node from the end of the list (the node with value 6) is removed.
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9

```

总结：就是链表的remove。

```

1  # This is an input class. Do not edit.
2  class LinkedList:
3      def __init__(self, value):
4          self.value = value
5          self.next = None
6
7
8  def removeKthNodeFromEnd(head, k):
9      # n is to count the number of the list.
10     n = 0
11     node = head
12     while node.next is not None:
13         n += 1
14         node = node.next
15     print(n)
16     remove_n_former = n - k
17     if remove_n_former < 0:

```

```

18         #直接改变head里面的构造值即可。
19         head.value = head.next.value
20         head.next = head.next.next
21     else:
22         node2 = head
23         while remove_n_former:
24             node2 = node2.next
25             remove_n_former -= 1
26         node3 = node2.next
27         node2.next = node3.next
28         node3.next = None
29

```

Sum Of LinkedLists

Sample Input

```

linkedListOne = 2 -> 4 -> 7 -> 1
linkedListTwo = 9 -> 4 -> 5

```

Sample Output

```

1 -> 9 -> 2 -> 2
// linkedListOne represents 1742
// linkedListTwo represents 549
// 1742 + 549 = 2291

```

```

1 # This is an input class. Do not edit.
2 class LinkedList:
3     def __init__(self, value):
4         self.value = value
5         self.next = None
6
7
8 def sumOfLinkedLists(linkedListOne, linkedListTwo):
9     # Write your code here.

```



```

10     newLinkedListHeadPointer = LinkedList(0)
11
12     currentNode = newLinkedListHeadPointer
13     carry = 0
14
15     nodeOne = linkedListOne
16     nodeTwo = linkedListTwo
17
18     while nodeOne is not None or nodeTwo is not None or carry != 0:
19         valueOne = nodeOne.value if nodeOne is not None else 0
20         valueTwo = nodeTwo.value if nodeTwo is not None else 0
21         sumOfValues = valueOne + valueTwo + carry
22
23         newValue = sumOfValues % 10
24         newNode = LinkedList(newValue)
25         currentNode.next = newNode
26         currentNode = newNode
27
28         carry = sumOfValues // 10
29         nodeOne = nodeOne.next if nodeOne is not None else None
30         nodeTwo = nodeTwo.next if nodeTwo is not None else None
31     return newLinkedListHeadPointer.next
32
33 #自己方法写的 思路也比较简单，就是通过遍历得出和相加，最后通过2个指针，重新构成链表即可。
34 # This is an input class. Do not edit.
35 class LinkedList:
36     def __init__(self, value):
37         self.value = value
38         self.next = None
39
40
41 def sumOfLinkedLists(linkedListOne, linkedListTwo):
42     # Write your code here.
43     p = linkedListOne
44     sum_p=sum_of_linklist(p,1,0)
45     print(sum_p)
46     q = linkedListTwo
47     sum_q = sum_of_linklist(q,1,0)
48     total_sum = sum_p+sum_q

```

```

49     print(total_sum // 1000)
50     multiply = 10
51     new_linked_list = LinkedList(total_sum % multiply)
52     s = new_linked_list
53     m = new_linked_list
54     total_sum = total_sum // multiply
55     while total_sum:
56         t = LinkedList(total_sum % multiply)
57         total_sum = total_sum // multiply
58         s.next = t
59         s = s.next
60     return m
61
62
63 def sum_of_linklist(link_list,multiplier,sum_list_value):
64     while link_list is not None:
65         sum_list_value += link_list.value*multiplier
66         multiplier *= 10
67         link_list = link_list.next
68     return sum_list_value

```

Get Permutations

Write a function that takes in an array of unique integers and returns an array of all permutations of those integers in no particular order.

If the input array is empty, the function should return an empty array.

Sample Input

```
array = [1, 2, 3]
```

Sample Output

```
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

Permutations

```

helper(arr, perm, perms):
    if arr is empty:
        perms.append(perm)

```

```

    else:
        for num in arr:
            newArr = removeNumFrom(arr)
            newPerm = perm + num
            helper(newArr, newPerm, perms)

```

[1, 2, 3]

[1, 2, 3]

[1, 3, 2]

[2, 1, 3]

[2, 3, 1]

[3, 2, 1]

[3, 1, 2]

algoexpert.io

Permutations

```

helper(arr, perm, perms):

```

```

    if arr is empty:
        perms.append(perm)

```

```

    else:

```

```

        for num in arr:
            newArr = removeNumFrom(arr)
            newPerm = perm + num
            helper(newArr, newPerm, perms)

```

[1, 2, 3]

[1, 2, 3]

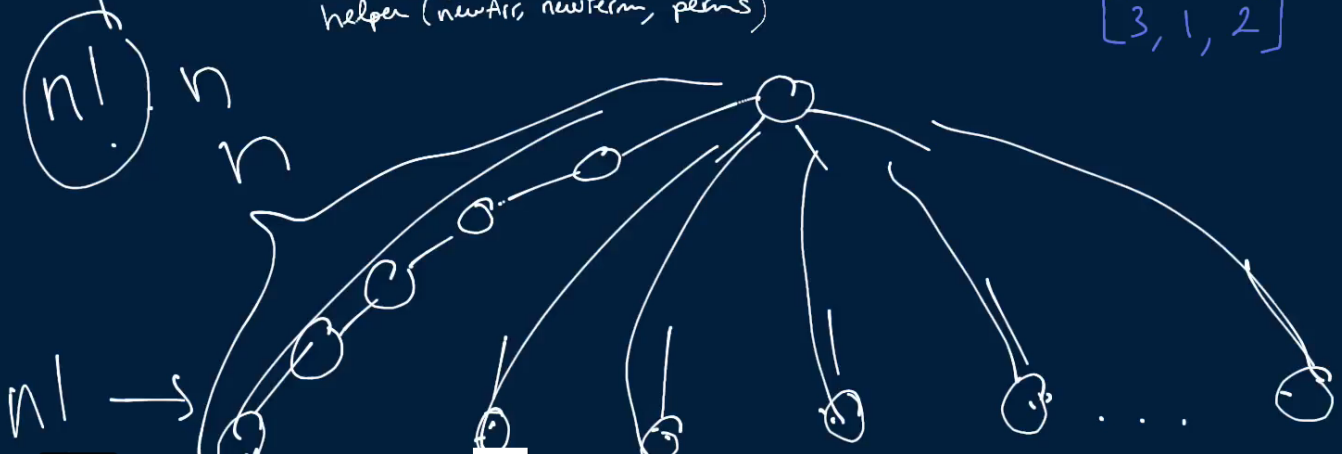
[1, 3, 2]

[2, 1, 3]

[2, 3, 1]

[3, 2, 1]

[3, 1, 2]



Permutations

helper(arr, perm, perms):

if arr is empty:

perms.append(perm)

else:

for num in arr:

newArr = removeNumFrom(arr)

newPerm = perm + num

helper(newArr, newPerm, perms)

$n!$ n n

[1, 2, 3]

[1, 2, 3]

[1, 3, 2]

[2, 1, 3]

[2, 3, 1]

[3, 2, 1]

[3, 1, 2]

Time = $O(N^2 N!)$

Space = $O(N \cdot N!)$

总结：第一个 $N!$ 表示最终有 $N!$ 的结果，第二个 N 表示每个结果实际里面有 N 个数字构成（即构成一个结果需要进行 N 次的回调），第三个 N 表示从原数列取出一个数，并重新构成一个数的时间复杂度。

helper(i, arr, perms):

if i last position:

perms.append(copy of arr)

else:

for j from i to end of arr:

swap(i, j)

helper(i+1, arr, perms)

swap(i, j)

第二个时间复杂度为 $O(N! \cdot N)$

```
1 def getPermutations(array):  
2     # Write your code here.  
3  
4     permutations = []  
5     permutationsHelper(array, [], permutations)
```

```

6     return permutations
7
8 def permutationsHelper(array,currentPermutation,permutations):
9     if not len(array) and len(currentPermutation):
10         permutations.append(currentPermutation)
11     else:
12         for i in range(len(array)):
13             newArray = array[:i] + array[i+1:]
14             newPermutation = currentPermutation +[array[i]]
15             permutationsHelper(newArray,newPermutation,permutations)
16
17 def getPermutations(array):
18     permutations = []
19     permutationsHelper(0,array,permutations)
20     return permutations
21
22 def permutationsHelper(i,array,permutations):
23     if i == len(array)-1 :
24         permutations.append(array[:])
25     else:
26         for j in range(i,len(array)):
27             swap(array,i,j)
28             permutationsHelper(i+1,array,permutations)
29             swap(array,i,j)
30
31 def swap(array,i,j):
32     array[i],array[j] = array[j],array[i]
33

```

Powerset

The powerset $P(X)$ of a set X is the set of all subsets of X . For example, the powerset of $[1,2]$ is $[[], [1], [2], [1,2]]$.

Note that the sets in the powerset do not need to be in any particular order.

Sample Input

```
array = [1, 2, 3]
```

Sample Output

```
[[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]
```

总结：仔细观察 $[2] = [] + [2]$; $[1,2] = [1] + [2]$; 从array中找到新的元素，然后不断添加到前面已经找到的powerset中即可。

```
1 def powerset(array):
2     subsets = [[]]
3     for ele in array:
4         for i in range(len(subsets)):
5             currentSubset = subsets[i]
6             subsets.append(currentSubset + [ele])
7     return subsets
```

Phone Number Mnemonics

If you open the keypad of your mobile phone, it'll likely look like this:

1	2	3
	abc	def

4	5	6
ghi	jkl	mno

7	8	9
pqrs	tuv	wxyz

Sample Input

```
phoneNumber = "1905"
```

Sample Output

```
[  
  "1w0j",  
  "1w0k",  
  "1w0l",  
  "1x0j",  
  "1x0k",  
  "1x0l",  
  "1y0j",  
  "1y0k",  
  "1y0l",  
  "1-z0",  
]
```

总结：建立一个字典，用来表示phonenumbers的索引结果。同时确定好结束的条件，即遍历到对应输入的数字长度时，将当前的结果添加进去即可。然后重复递归。

```

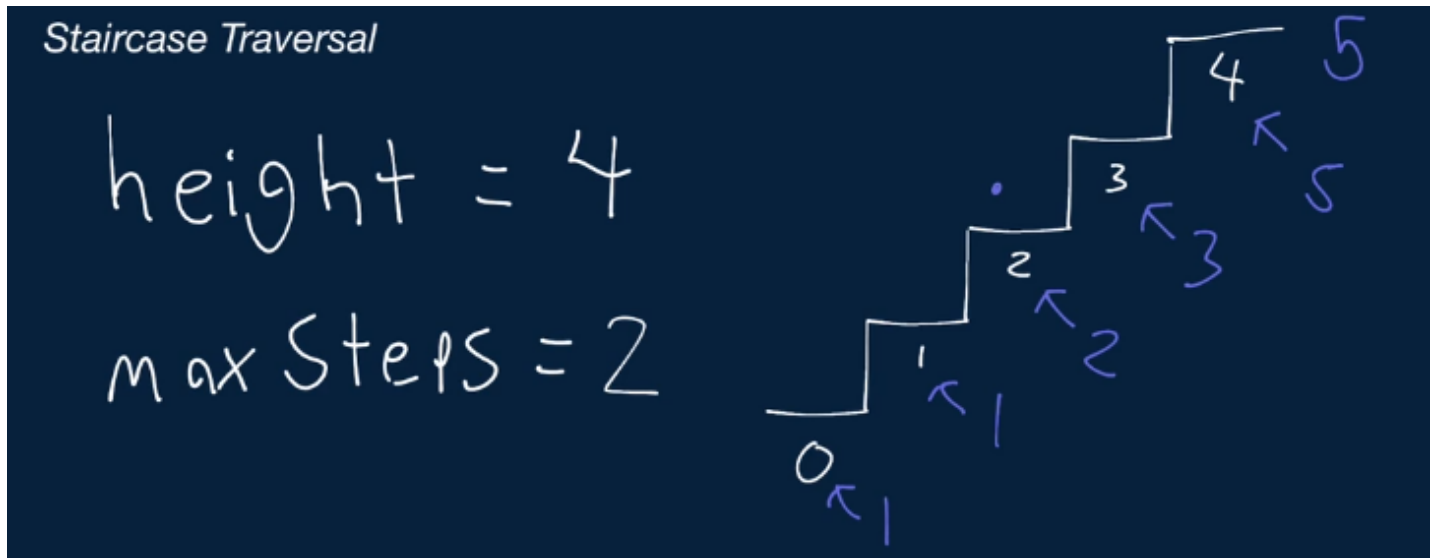
1 def phoneNumberMnemonics(phoneNumber):
2     currentMnemonic = ["0"] * len(phoneNumber)
3
4     print(currentMnemonic )
5     phoneNumberMnemonicsHelper(0,phoneNumber,currentMnemonic,mnemonicsFound)
6
7     return mnemonicsFound
8
9 def phoneNumberMnemonicsHelper(idx,phoneNumber,currentMnemonic,mnemonicsFound):
10    #最后结束的时候
11    if idx == len(phoneNumber):
12        mnemonic = "".join(currentMnemonic) #O(n)
13        mnemonicsFound.append(mnemonic)
14    else:
15        digit = phoneNumber[idx]
16        letters = DIGIT_LETTERS[digit]
17        for letter in letters:
18            #用于存储单个遍历结果的列表，设定好之后使用更稳定准确。
19            currentMnemonic[idx] = letter
20            #如果还有继续寻找下一个recursive function
21            phoneNumberMnemonicsHelper(idx+1,phoneNumber,currentMnemonic,mnemonicsFound)
22
23
24
25 DIGIT_LETTERS = {
26     "0": ["0"],
27     "1": ["1"],
28     "2": ["a", "b", "c"],
29     "3": ["d", "e", "f"],
30     "4": ["g", "h", "i"],
31     "5": ["j", "k", "l"],
32     "6": ["m", "n", "o"],
33     "7": ["p", "q", "r", "s"],
34     "8": ["t", "u", "v"],
35     "9": ["w", "x", "y", "z"],
36 }
37

```

Staircase Traversal

For example, if you were given a staircase of `height = 3` and `maxSteps = 2` you could climb the staircase in 3 ways. You could take **1 step, 1 step, then 1 step**, you could also take **1 step, then 2 steps**, and you could take **2 steps, then 1 step**.

Note that `maxSteps <= height` will always be true.



总结：第4层的5 等于 height - maxSteps 的2个层数的求和。2+3 。 用一个字典来进行遍历，初始0层为1,1层为1，作为递归的终结条件。

```
1 def staircaseTraversal(height, maxSteps):
2     return numberOfWaysToTop(height, maxSteps, {0:1, 1:1})
3
4 def numberOfWaysToTop(height, maxSteps, memoize):
5     if height in memoize:
6         return memoize[height]
7
8     numberOfWays = 0
9
10    for step in range(1, min(maxSteps, height)+1):
11        numberOfWays += numberOfWaysToTop(height-step, maxSteps, memoize)
12
13    memoize[height] = numberOfWays
14
15    return numberOfWays
```

Search In Sorted Matrix

You're given a two-dimensional array (a matrix) of distinct integers and a target integer. Each row in the matrix is sorted, and each column is also sorted; the matrix doesn't necessarily have the same height and width.

Write a function that returns an array of the row and column indices of the target integer if it's contained in the matrix, otherwise `[-1, -1]`.

Sample Input

Sample Input

```
matrix = [  
  [1, 4, 7, 12, 15, 1000],  
  [2, 5, 19, 31, 32, 1001],  
  [3, 8, 24, 33, 35, 1002],  
  [40, 41, 42, 44, 45, 1003],  
  [99, 100, 103, 106, 128, 1004],  
]  
target = 44
```

Sample Output

```
[3, 3]
```

总结：注意题目的条件，从右上角找最好。如果发现该数>target则 col -= 1 如果发现该数小于target，则row += 1，即可。

```
1 def searchInSortedMatrix(matrix, target):  
2     row = 0  
3     col = len(matrix[0]) - 1  
4     #建构一个必须的条件即可  
5     while row < len(matrix) and col >= 0:  
6         if matrix[row][col] > target:  
7             col -= 1  
8         elif matrix[row][col] < target:  
9             row += 1  
10        else:  
11            return [row, col]  
12    return [-1, -1]
```

Three Number Sort

Sample Input

```
array = [1, 0, 0, -1, -1, 0, 1, 1]
order = [0, 1, -1]
```

Sample Output

```
[0, 0, 0, 1, 1, 1, -1, -1]
```

总结：对原数列通过字典构建一个元素和对应的数量。然后进行还原即可。

```
1 def threeNumberSort(array, order):
2     #establish a dict that contains the number of the Num.
3     number_dict = {}
4     if len(array) != 0:
5         for i in range(len(array)):
6             if array[i] not in number_dict:
7                 number_dict[array[i]] = 1
8             else:
9                 number_dict[array[i]] += 1
10
11     #according to the order to reorder the array
12     order_array = []
13     for i in range(len(order)):
14         if order[i] not in number_dict:
15             continue
16         for j in range(number_dict[order[i]]):
17             order_array.append(order[i])
18     else:
19         order_array = []
20     return order_array
21
```

Min Max Stack (最大最小栈)

Min Max Stack Construction

Write a `MinMaxStack` class for a Min Max Stack. The class should support:

- Pushing and popping values on and off the stack.
- Peeking at the value at the top of the stack.
- Getting both the minimum and the maximum values in the stack at any given point in time.

All class methods, when considered independently, should run in constant time and with constant space.

总结：通过python [] 直接构建 最大最小栈

```
1 # Feel free to add new properties and methods to the class.
2 class MinMaxStack:
3     def __init__(self):
4         self.MinMaxstack = []
5     def peek(self):
6         return self.MinMaxstack[len(self.MinMaxstack)-1]
7     def pop(self):
8         pop_n = self.MinMaxstack.pop()
9         return pop_n
10
11     def push(self, number):
12         self.MinMaxstack.append(number)
13
14     def getMin(self):
15         return min(self.MinMaxstack)
16
17     def getMax(self):
18         return max(self.MinMaxstack)
19
```

semordnilap (寻找倒装的数)

Semordnilap 🟢 ☆

Write a function that takes in a list of unique strings and returns a list of semordnilap pairs.

A semordnilap pair is defined as a set of different strings where the reverse of one word is the same as the forward version of the other. For example the words "diaper" and "repaid" are a semordnilap pair, as are the words "palindromes" and "semordnilap".

The order of the returned pairs and the order of the strings within each pair does not matter.

Sample Input

```
words = ["diaper", "abc", "test", "cba", "repaid"]
```

Sample Output

```
[["diaper", "repaid"], ["abc", "cba"]]
```

思路：通过集合，将重复的元素删除掉，然后通过for循环进行遍历，判断是否是倒装，并将发现的元素从集合中删除。

```
1 def semordnilap(words):
2
3     #去除重复的元素
4     wordSet = set(words)
5     semordnilapPairs = []
6
7     #遍历寻找，并判断是否为倒装
8     for word in words:
9         reversedWord = word[::-1]
10        if reversedWord in wordSet and reversedWord != word:
11            semordnilapPairs.append([word, reversedWord])
12            wordSet.remove(word)
13            wordSet.remove(reversedWord)
14
15    return semordnilapPairs
```

balanced Brackets (平衡的括号)

Balanced Brackets 🟢 ☆

Write a function that takes in a string made up of brackets (`(`, `[`, `{`, `)`, `]`, and `}`) and other optional characters. The function should return a boolean representing whether the string is balanced with regards to brackets.

A string is said to be balanced if it has as many opening brackets of a certain type as it has closing brackets of that type and if no bracket is unmatched. Note that an opening bracket can't match a corresponding closing bracket that comes before it, and similarly, a closing bracket can't match a corresponding opening bracket that comes after it. Also, brackets can't overlap each other as in `[ ( ) ]`.

Sample Input

```
string = "([ ]) () {} ( ) () ()"
```

Sample Output

```
true // it's balanced
```

思路：构建列表对左括号进行存储，当发现右括号时，对应的左括号里面的字典数字减1。当数字小于0时，则直接判断为False。最后做一个总的判断，要求字典里的所有元素都为0。

```
1 def balancedBrackets(string):
2     dict_left_brackets = {}
3     dict_left_brackets['('] = 0
4     dict_left_brackets['['] = 0
5     dict_left_brackets['{'] = 0
6     is_balanced = True
7     for char in string:
8         if char in dict_left_brackets:
9             dict_left_brackets[char] += 1
10        elif char == ')':
11            dict_left_brackets['('] -= 1
12            if dict_left_brackets['('] < 0 :
13                is_balanced = False
14                break
15        elif char == ']':
16            dict_left_brackets['['] -= 1
```

```

17         if dict_left_brackets['['] < 0 :
18             is_balanced = False
19             break
20     elif char == '}':
21         dict_left_brackets['{'] -= 1
22         if dict_left_brackets['{'] < 0 :
23             is_balanced = False
24             break
25 for i in dict_left_brackets:
26     if dict_left_brackets[i] != 0:
27         is_balanced = False
28         break
29 return is_balanced

```

sunset Views（水平线视角【根据方向寻找逐步的最大值】）

Given an array of buildings and a direction that all of the buildings face, return an array of the indices of the buildings that can see the sunset.

A building can see the sunset if it's strictly taller than all of the buildings that come after it in the direction that it faces.

The input array named `buildings` contains positive, non-zero integers representing the heights of the buildings. A building at index `i` thus has a height denoted by `buildings[i]`. All of the buildings face the same direction, and this direction is either east or west, denoted by the input string named `direction`, which will always be equal to either `"EAST"` or `"WEST"`. In relation to the input array, you can interpret these directions as right for east and left for west.

Important note: the indices in the output array should be sorted in ascending order.

总结：根据direction 判断遍历的方向，并确立一个临时的最大值存储即可。

```

1 def sunsetViews(buildings, direction):
2     sunsetViews = []
3     begin_i = 0 if direction == "WEST" else len(buildings) - 1
4     step = 1 if direction == "WEST" else -1
5
6     idx = begin_i
7     highest_buildings = 0
8
9     while idx >= 0 and idx <= len(buildings) - 1:
10         buildingHeight = buildings[idx]

```

```

11         if buildingHeight > highest_buildings:
12             sunsetViews.append(idx)
13
14             highest_buildings = max(highest_buildings, buildingHeight)
15
16             idx += step
17
18
19
20     if direction == "EAST":
21         return sunsetViews[::-1]
22     return sunsetViews

```

sort Stack （只用pop和append 栈的方法解决排序问题）

思路：循环遍历的方式，首先把所有数给pop出来。然后逐步放入，要求栈里的数要小于外面的数，否则，栈里的数先pop出来，直到可以放进去了。

Sample Input

```
stack = [-5, 2, -2, 4, 3, 1]
```

Sample Output

```
[-5, -2, 1, 2, 3, 4]
```

```

1 def sortStack(stack):
2     if len(stack) == 0:
3         return stack
4     top = stack.pop()
5     sortStack(stack)
6     insertOrderStack(stack, top)
7     return stack
8
9 def insertOrderStack(stack, value):
10    if len(stack) == 0 or stack[len(stack)-1] <= value:

```



```
11         stack.append(value)
12         return
13     top = stack.pop()
14     insertOrderStack(stack, value)
15     stack.append(top)
```

next Greater Element (找到当前位置后面离它最近比当前值大的数)

Write a function that takes in an array of integers and returns a new array containing, at each index, the next element in the input array that's greater than the element at that index in the input array.

In other words, your function should return a new array where `outputArray[i]` is the next element in the input array that's greater than `inputArray[i]`. If there's no such next greater element for a particular index, the value at that index in the output array should be `-1`. For example, given `array = [1, 2]`, your function should return `[2, -1]`.

Additionally, your function should treat the input array as a **circular** array. A circular array wraps around itself as if it were connected end-to-end. So the next index after the last index in a circular array is the first index. This means that, for our problem, given

`array = [0, 0, 5, 0, 0, 3, 0, 0]`, the next greater element after `3` is `5`, since the array is circular.

Sample Input

```
array = [2, 5, -3, -4, 6, 7, 2]
```

Sample Output

```
[5, 6, 6, 6, 7, -1, 5]
```

思路：思路不难，首先因为有个首尾循环，所以在找比当前数更大的数时，可以将原来的数容量扩大一倍，这样就方便了查找。如果找到了，则将当前数append到结果中，如果没找到，则添加-1。

```
1 def nextGreaterElement(array):
2     copy_array = array.copy()
3     array.extend(copy_array)
4
5     nextGreaterArray = []
```

```

6     for i in range(len(copy_array)):
7         find_flag = False
8         for j in range(i, len(array)):
9             if array[j] > copy_array[i]:
10                 find_flag = True
11                 nextGreaterArray.append(array[j])
12                 break
13         if find_flag == False:
14             nextGreaterArray.append(-1)
15
16     return nextGreaterArray

```

longest Palindromic Substring

Sample Input

```
string = "abaxyzzyxf"
```

Sample Output

```
"xyzzyx"
```

思路：寻找最长的首位相同的子链，可以通过i和j分别表示，子链的首位置和末尾位置。然后根据首尾相同的条件，构建另一个函数进行判断，即可。

```

1  def longestPalindromicSubstring(string):
2      # to find the same element and cut it from the string
3      if string is None:
4          return string
5      max_length = 0
6      for i in range(len(string)-max_length):
7          for j in range(i, len(string)):
8              if isPalindromic(string[i:j+1]) and (j-i+1)>max_length:
9                  max_length = j-i+1
10                 output_string = string[i:j+1]
11
12     return output_string

```

```

13
14
15 def isPalindromic(string):
16     begin = 0
17     end = len(string)-1
18     isPalindromic = True
19     while begin < end:
20         if string[begin] == string[end]:
21             begin += 1
22             end -= 1
23         else:
24             isPalindromic = False
25             break
26     return isPalindromic

```

group Anagrams

Group Anagrams 🟢 ☆

Write a function that takes in an array of strings and groups anagrams together.

Anagrams are strings made up of exactly the same letters, where order doesn't matter. For example, "cinema" and "iceman" are anagrams; similarly, "foo" and "ofo" are anagrams.

Your function should return a list of anagram groups in no particular order.

Sample Input

```
words = ["yo", "act", "flop", "tac", "foo", "cat", "oy", "olfp"]
```

Sample Output

```
[["yo", "oy"], ["flop", "olfp"], ["act", "tac", "cat"], ["foo"]]
```

思路：跟上面的题目类似，是上面的进阶版，需要对重复的元素进行判断，这里设置一个arrange的列表，用-1表示没有归类，1表示已经归类了。通过字典对两个元素进行判断，是否是anagrams。

```

1 def groupAnagrams(words):
2     arranged = [-1 for i in range(len(words))]
3     groupAnagrams = []
4
5     for i in range(len(words)):
6         subGroupAnagrams = [words[i]]
7         for j in range(i+1, len(words)):
8             if isAnagrams(words[i], words[j]) and arranged[j] == -1:
9                 subGroupAnagrams.append(words[j])
10                arranged[j] = 1
11            if arranged[i] == -1:
12                groupAnagrams.append(subGroupAnagrams)
13    return groupAnagrams
14
15 def isAnagrams(word1, word2):
16     dict_word = {}
17     isAnagrams = True
18     for each_word1 in word1:
19         if each_word1 not in dict_word:
20             dict_word[each_word1] = 1
21         else:
22             dict_word[each_word1] += 1
23
24     for each_word2 in word2:
25         if each_word2 not in dict_word:
26             isAnagrams = False
27             break
28         else:
29             dict_word[each_word2] -= 1
30     if isAnagrams:
31         for each_dict in dict_word:
32             if dict_word[each_dict] != 0:
33                 isAnagrams = False
34    return isAnagrams

```

valid IP Addresses

Sample Input

```
string = "1921680"
```

Sample Output

```
[  
    "1. 9. 216. 80",  
    "1. 92. 16. 80",  
    "1. 92. 168. 0",  
    "19. 2. 16. 80",  
    "19. 2. 168. 0",  
    "19. 21. 6. 80",  
    "19. 21. 68. 0",  
    "19. 216. 8. 0",  
    "192. 1. 6. 80",  
    "192. 1. 68. 0",  
    "192. 16. 8. 0"  
]
```

思路：用currentIPAddressParts = ["","","",""], 这个列表作为标准件，收留从string里面分好的ip段，然后不断进行判断每个段内的地址是否为合理的即可（有前置0的不可）。

```
1 def validIPAddresses(string):  
2     output_list = []  
3     for i in range(1,min(len(string),4)):  
4  
5         currentIPAddressParts = ["","","",""]  
6  
7         currentIPAddressParts[0] = string[:i]  
8  
9         if not isValidIPAddresses(currentIPAddressParts[0]):  
10             continue  
11  
12         for j in range(i + 1,i + min(len(string) - i, 4)):  
13             currentIPAddressParts[1] = string[i:j]  
14             if not isValidIPAddresses(currentIPAddressParts[1]):  
15                 continue  
16             for k in range(j + 1 ,j + min(len(string) - j, 4)):  
17                 currentIPAddressParts[2] = string[j:k]  
18                 currentIPAddressParts[3] = string[k:]  
19  
20                 if isValidIPAddresses(currentIPAddressParts[3]) and isValidIP
```

```

20         output_list.append(".".join(currentIPAddressParts))
21
22     return output_list
23
24 def isValidIPAddresses(array):
25     stringAsInt = int(array)
26     if stringAsInt > 255:
27         return False
28     return len(array) == len(str(stringAsInt)) #check for leading 0

```

reverse Words In String

Reverse Words In String 🟢 ☆

Write a function that takes in a string of words separated by one or more whitespaces and returns a string that has these words in reverse order. For example, given the string `"tim is great"`, your function should return `"great is tim"`.

For this problem, a word can contain special characters, punctuation, and numbers. The words in the string will be separated by one or more whitespaces, and the reversed string must contain the same whitespaces as the original string. For example, given the string `"whitespaces 4"` you would be expected to return `"4 whitespaces"`.

Note that you're **not** allowed to use any built-in `split` or `reverse` methods/functions. However, you **are** allowed to use a built-in `join` method/function.

Also note that the input string isn't guaranteed to always contain words.

Sample Input

```
string = "AlgoExpert is the best!"
```

Sample Output

```
"best! the is AlgoExpert"
```

思路：主要关键点在于如何找到词还是空格，空格作为单独的一个元素添加到word即可，词需要一个判断。

```
1 def reverseWordsInString(string):
```

```

2  words = []
3  begin_word = 0
4  for idx in range(len(string)):
5      character = string[idx]
6
7      if character == " ":
8          words.append(string[begin_word:idx])
9          begin_word = idx
10
11         elif string[begin_word] == " ":
12             words.append(" ")
13             begin_word = idx
14
15     words.append(string[begin_word:])
16
17     reverseList(words)
18
19     return "".join(words)
20
21 def reverseList(list):
22     start,end = 0,len(list)-1
23     while start < end :
24         list[start],list[end] = list[end],list[start]
25         start += 1
26         end -= 1
27

```

zero Sum Subarray

Zero Sum Subarray 🟢 ☆

You're given a list of integers `nums`. Write a function that returns a boolean representing whether there exists a zero-sum subarray of `nums`.

A zero-sum subarray is any subarray where all of the values add up to zero. A subarray is any contiguous section of the array. For the purposes of this problem, a subarray can be as small as one element and as long as the original array.

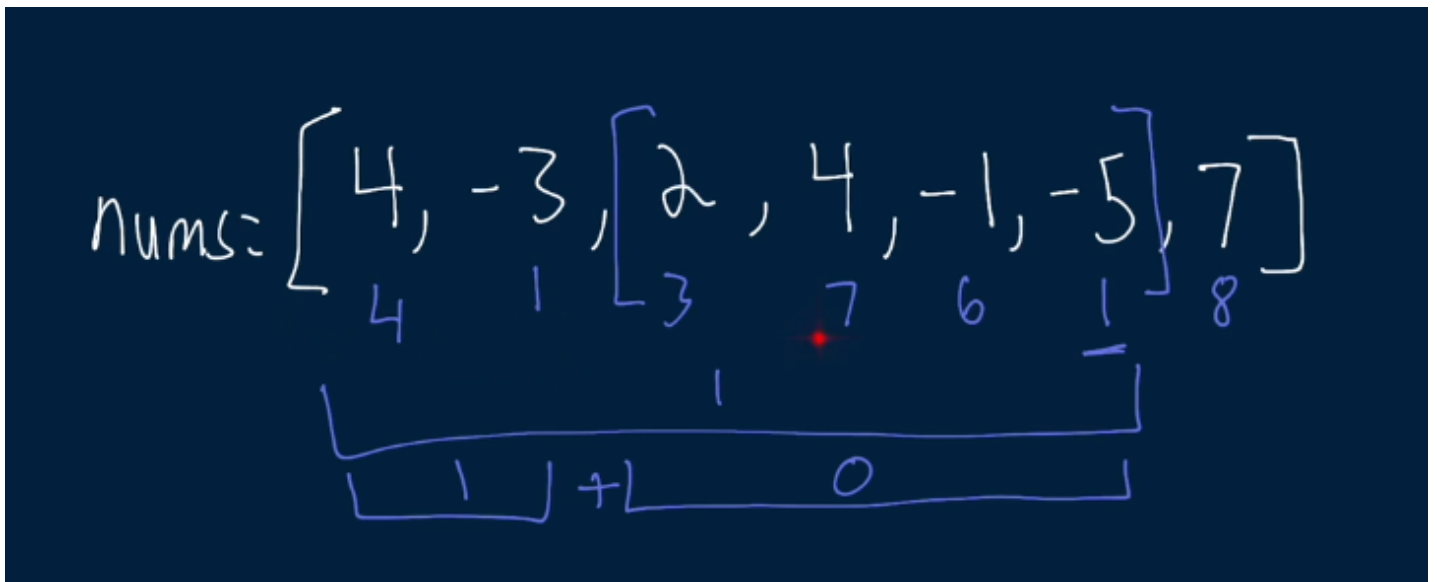
Sample Input

```
nums = [-5, -5, 2, 3, -2]
```

Sample Output

```
True // The subarray [-5, 2, 3] has a sum of 0
```

思路：自己通过遍历查找子链，判断是否有为0的。官方的思路：



```
1 def zeroSumSubarray(nums):
2     if len(nums) == 1:
3         if nums[0] == 0:
4             return True
5     else:
6         for i in range(len(nums)-1):
7             for j in range(i, len(nums)):
8                 if isZeroSum(i, j, nums):
9                     return True
10    return False
11
12 def isZeroSum(i, j, nums):
```



```
13     sum = 0
14     if j == len(nums) - 1:
15         for each_element in nums[i:]:
16             sum += each_element
17     else:
18         for each_element in nums[i:j+1]:
19             sum += each_element
20     #print(f"i is {i}")
21     #print(f"j is {j}")
22     #print(sum)
23
24
25     if sum == 0:
26         return True
27     else:
28         return False
29
30
31 def zeroSumSubarray(nums):
32     sums = set([0])
33     currentSum = 0
34     for num in nums:
35         currentSum += num
36         if currentSum in sums:
37             return True
38         sums.add(currentSum)
39
40     return False
41
```

minimum Characters For Words

Sample Input

```
words = ["this", "that", "did", "deed", "them!", "a"]
```

Sample Output

```
["t", "t", "h", "i", "s", "a", "d", "d", "e", "e", "m", "!"]  
// The characters could be ordered differently.
```

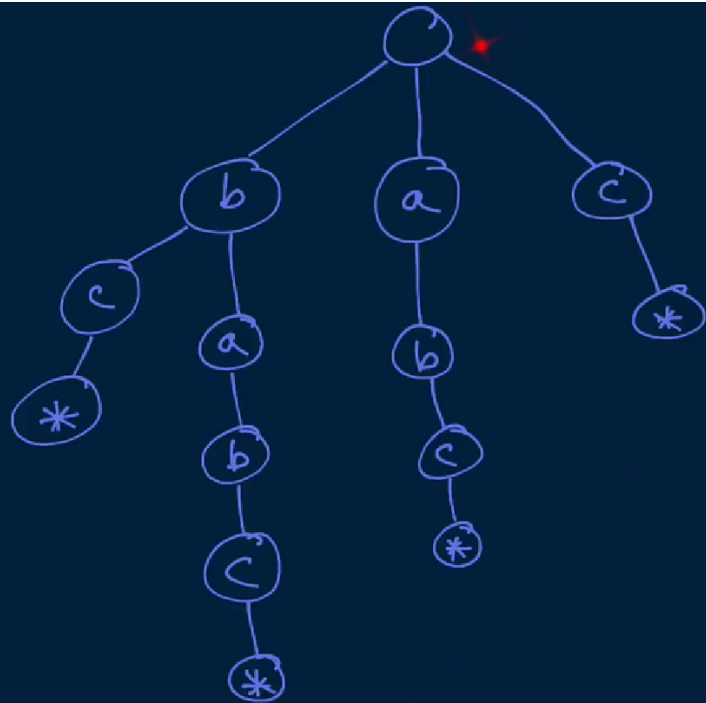
思路：根据word里面的不同元素，构建一个dict，然后判断所有元素里面的字典数据，将最大值更新到输出字典中，再将输出字典换算成列表即可。

```
1 def minimumCharactersForWords(words):  
2     characters_list = []  
3     output_dict = {}  
4     output_list = []  
5     for word in words:  
6         minimum_characters_dict = {}  
7         for each_element in word:  
8             if each_element not in minimum_characters_dict:  
9                 minimum_characters_dict[each_element] = 1  
10            else:  
11                minimum_characters_dict[each_element] += 1  
12            characters_list.append(minimum_characters_dict)  
13        print(characters_list)  
14        #作者的方法是建立一个函数进行实时地比较  
15        for character_dict in characters_list:  
16            for each_character in character_dict:  
17                if each_character not in output_dict:  
18                    output_dict[each_character] = character_dict[each_character]  
19                else:  
20                    output_dict[each_character] = max(character_dict[each_character], output_dict[each_character])  
21        print(output_dict)  
22        for each_element in output_dict:  
23            for i in range(output_dict[each_element]):  
24                output_list.append(each_element)  
25    return output_list
```

Suffix Trie

Suffix Trie Construction

b a b c



```
string = "babc"
```

Sample Output (for creation)

The structure below is the root of the trie.

```
{
  "c": {"*": true},
  "b": {
    "c": {"*": true},
    "a": {"b": {"c": {"*": true}}},
  },
  "a": {"b": {"c": {"*": true}}},
}
```

Sample Input (for searching in the suffix trie above)

```
string = "abc"
```

Sample Output (for searching in the suffix trie above)

```
true
```

思路：题目的意思就是将左边的字符串，变成右边的suffix trie即可。
方法通过类似构建字典的方式，逐层套娃。

```
1 class SuffixTrie:
2     def __init__(self, string):
3         self.root = {}
4         self.endSymbol = "*"
5         self.populateSuffixTrieFrom(string)
6
7     def populateSuffixTrieFrom(self, string):
8
9         for i in range(len(string)):
10             self.insertSubstringStartingAt(i, string)
11
12     def insertSubstringStartingAt(self, i, string):
13         node = self.root
14         for j in range(i, len(string)):
15             letter = string[j]
16             if letter not in node:
17                 node[letter] = {}
18             node = node[letter]
19         node[self.endSymbol] = True
20
21
22     def contains(self, string):
23         node = self.root
24         for letter in string:
25             if letter not in node:
26                 return False
```

```
27         node = node[letter]
28     return self.endSymbol in node
```

Merge Binary Trees

Merge Binary Trees 🟢 ☆

Given two binary trees, merge them and return the resulting tree. If two nodes overlap during the merger then sum the values, otherwise use the existing node.

Note that your solution can either mutate the existing trees or return a new tree.

Sample Input

```
tree1 =      1
           /  \
          3    2
         /  \
        7    4

tree2 =      1
           /  \
          5    9
         /  \  /  \
        2   7 6
```

Sample Output

```
output =     2
           /  \
          8    11
         /  \  /  \
        9   4 7   6
```

思路: 通过递归的方式解决该问题, 首先设置好最后的返回条件, 当tree1为None时, 则直接返回tree2 同理, tree2为None时, 则直接返回tree1。然后便是对应位置的求和, 递归循环的开始, 即tree1和tree2的左节点和左节点相加, 右节点和右节点相加。并将最终结果存储在tree1中。最后返回。

思路: 设置好结束的条件, 当tree1遍历到None节点, 直接返回tree2, 同理tree2。然后让tree1.value加上对应位置tree2.value。然后就是递归, 递归结束, 返回tree1, 即可。

```
1 # This is an input class. Do not edit.
2 class BinaryTree:
```

```

3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7
8
9     def mergeBinaryTrees(tree1, tree2):
10        if tree1 is None:
11            return tree2
12        if tree2 is None:
13            return tree1
14
15        tree1.value += tree2.value
16        tree1.left = mergeBinaryTrees(tree1.left, tree2.left)
17        tree1.right = mergeBinaryTrees(tree1.right, tree2.right)
18        return tree1

```

Stable Internships

Stable Internships

$[[0,1],[1,0],[2,2]]$

interns: [

- [0,1,2],
- [0,2,1],
- [1,2,0]

]

teams: [

- [2,1,0],
- [0,1,2],
- [0,1,2],

]

$[[0,1],[1,0],[2,2]]$

思路：题目的意思大致是给2个列表，一个是放interns中意的team，另一个是team中意的interns。以interns的中意度为优先。首先根据interns的中意度最优先的构成匹配表，然后对产生冲突的team，根据team的优先度，进行修正，然后再根据冲突后匹配失败的intern的中意度，匹配下一个team，直到冲突没了。

Stable Internships

$[[0,1],[1,0],[2,2]]$

interns: [
 [0,1,2],
 [0,2,1],
 [1,2,0]
]

teams: [
 [2,1,0],
 [0,1,2],
 [0,1,2],
]

Chosen Interns: { 3
 free Interns: [0, 1, 2]
 current Intern Choices: [0, 0, 0]

team Maps: [
 {2:0, 1:1, 0:2},
 {0:0, 1:1, 2:2},
 {0:0, 1:1, 2:2}
]

Stable Internships

$[[0,1],[1,0],[2,2]]$

interns: [
~~[0,1,2]~~,
~~[0,2,1]~~,
~~[1,2,0]~~
]

teams: [
 [2,1,0],
 [0,1,2],
 [0,1,2],
]

Chosen Interns: {1:0, 0:1, 2:2} 3
 free Interns: [
]
 current Intern Choices: [2, 1, 2]

team Maps: [
 {2:0, 1:1, 0:2},
 {0:0, 1:1, 2:2},
 {0:0, 1:1, 2:2}
]

上图是逻辑实现过程，currentInternChoice是下标。下面是对应的代码逻辑：

```
1 def stableInternships(interns, teams):
2     chosenInterns = {}
```

```

3     freeInterns = list(range(len(interns)))
4     currentInternChoices = [0] * len(interns)
5
6     #给teams中按照interns进行优先级排序
7     teamMaps = []
8     for team in teams:
9         rank = {}
10        for i,internNum in enumerate(team):
11            rank[internNum] = i
12        teamMaps.append(rank)
13
14
15    while len(freeInterns) > 0:
16        internNum = freeInterns.pop()
17
18        intern = interns[internNum]
19        teamPreference = intern[currentInternChoices[internNum]]
20        currentInternChoices[internNum] += 1
21
22        if teamPreference not in chosenInterns:
23            chosenInterns[teamPreference] = internNum
24            continue
25        previousIntern = chosenInterns[teamPreference]
26        previousInternRank = teamMaps[teamPreference][previousIntern]
27        currentInternRank = teamMaps[teamPreference][internNum]
28
29        if currentInternRank < previousInternRank:
30            freeInterns.append(previousIntern)
31            chosenInterns[teamPreference] = internNum
32        else:
33            freeInterns.append(internNum)
34
35    matches = [[internNum,teamNum] for teamNum,internNum in chosenInterns.items()]
36    return matches

```

Two Colorable

Sample Input

```
edges = [
    [1, 2],
    [0, 2],
    [0, 1]
]
```

Sample Output

Nodes 1 and 2 must be different colors than node 0.
However, nodes 1 and 2 are also connected, meaning they must also have different colors,
which is impossible with only 2 available colors.

思路：就是整个图只能出现2种颜色，两个相连接的节点的颜色必须不一样，否则返回false。

通过一个栈，按照图的链接进行按图索骥，如果还没有安排，则将该点设置为当前节点的反例（表示颜色不一样，本题为false），如果发现有个节点已经安排过了，则判断其颜色是否和对应链接的节点颜色是否一样，一样则返回False。

```
1 def twoColorable(edges):
2     arranged_edges = [None for _ in edges]
3     arranged_edges[0] = True
4     stack = [0]
5
6     while len(stack)>0:
7         node = stack.pop()
8         for connection in edges[node]:
9             if arranged_edges[connection] is None:
10                 arranged_edges[connection] = not arranged_edges[node]
11                 stack.append(connection)
12             elif arranged_edges[connection] == arranged_edges[node]:
13                 return False
14
15     return True
```

Zero Sum Subarray

Zero Sum Subarray

You're given a list of integers `nums`. Write a function that returns a boolean representing whether there exists a zero-sum subarray of `nums`.

A zero-sum subarray is any subarray where all of the values add up to zero. A subarray is any contiguous section of the array. For the purposes of this problem, a subarray can be as small as one element and as long as the original array.

Sample Input

```
nums = [-5, -5, 2, 3, -2]
```

Sample Output

```
True // The subarray [-5, 2, 3] has a sum of 0
```

思路:通过2个for循环, 进行子列的遍历获取, 然后判断子列的和是否为0, 如果则返回true, 否则继续遍历, 直到发现和为0的子列, 返回true, 否则返回false。

```
1 def zeroSumSubarray(nums):
2     if len(nums) == 1:
3         if nums[0] == 0:
4             return True
5     else:
6         for i in range(len(nums)-1):
7             for j in range(i, len(nums)):
8                 if isZeroSum(i, j, nums):
9                     return True
10    return False
11
12 def isZeroSum(i, j, nums):
13     sum = 0
14     if j == len(nums) - 1:
15         for each_element in nums[i:]:
16             sum += each_element
17     else:
18         for each_element in nums[i:j+1]:
19             sum += each_element
```

```

20     #print(f"i is {i}")
21     #print(f"j is {j}")
22     #print(sum)
23
24
25     if sum == 0:
26         return True
27     else:
28         return False
29

```

MergingLinkedLists

You're given two Linked Lists of potentially unequal length. These Linked Lists potentially merge at a shared intersection node. Write a function that returns the intersection node or returns `None` / `null` if there is no intersection.

Each `LinkedList` node has an integer `value` as well as a `next` node pointing to the next node in the list or to `None` / `null` if it's the tail of the list.

Note: Your function should return an existing node. It should not modify either Linked List, and it should not create any new Linked Lists.

Sample Input

```

linkedListOne = 2 -> 3 -> 1 -> 4
linkedListTwo = 8 -> 7 -> 1 -> 4

```

Sample Output

```

1 -> 4 // The lists intersect at the node with value 1

```

思路:这题只有1个交叉点，而且只要交叉了之后就会一样。所以遍历找到对应相同的元素即可。

```

1 # This is an input class. Do not edit.
2 class LinkedList:
3     def __init__(self, value):
4         self.value = value
5         self.next = None
6
7 #该题只有1个交集点，要么有要么没有
8 def mergingLinkedLists(linkedListOne, linkedListTwo):

```

```

9     listOneNodes = set()
10    currentNodeOne = linkedListOne
11    while currentNodeOne is not None:
12        listOneNodes.add(currentNodeOne)
13        currentNodeOne = currentNodeOne.next
14
15    currentNodeTwo = linkedListTwo
16    while currentNodeTwo is not None:
17        if currentNodeTwo in listOneNodes:
18            return currentNodeTwo
19        currentNodeTwo = currentNodeTwo.next
20
21    return None

```

SymmetricalTree

Symmetrical Tree

Write a function that takes in a Binary Tree and returns if that tree is symmetrical. A tree is symmetrical if the left and right subtrees are mirror images of each other.

Each `BinaryTree` node has an integer `value`, a `left` child node, and a `right` child node. Children nodes can either be `BinaryTree` nodes themselves or `None` / `null`.

Sample Input

```

tree =      1
           /  \
          2    2
         / \  / \
        3  4 4  3
       / \ / \ / \
      5  6 6  5

```

Sample Output

True

思路:通过建立两个栈（列表），分别储存对应树的左右子节点。然后分别取出来判断是否是空节点或者是否相等，如果不相等，则返回false。

```

1 # This is an input class. Do not edit.

```

```

2 class BinaryTree:
3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7
8
9 def s(tree):
10     stackLeft = [tree.left]
11     stackRight = [tree.right]
12
13     while len(stackLeft) > 0:
14         left = stackLeft.pop()
15         right = stackRight.pop()
16
17         if left is None and right is None:
18             continue
19         if left is None or right is None or left.value != right.value:
20             return False
21
22         stackLeft.append(left.left)
23         stackLeft.append(left.right)
24         stackRight.append(right.right)
25         stackRight.append(right.left)
26
27     return True

```

One Edit

You're given two strings `stringOne` and `stringTwo`. Write a function that determines if these two strings can be made equal using only one edit.

There are 3 possible edits:

- **Replace:** One character in one string is swapped for a different character.
- **Add:** One character is added at any index in one string.
- **Remove:** One character is removed at any index in one string.

Note that both strings will contain at least one character. If the strings are the same, your function should return true.

Sample Input

```
stringOne = "hello"  
stringTwo = "hollo"
```

Sample Output

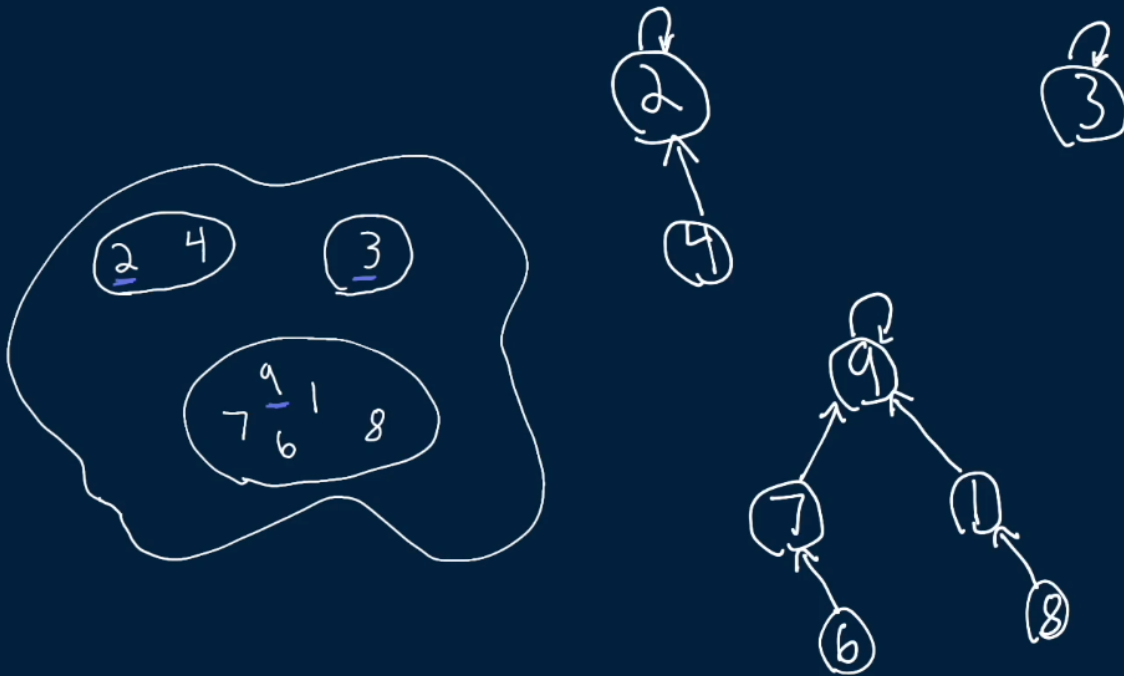
```
True // A single replace at index 1 of either string can make the strings equal
```

思路:增加和删除本质都是差不多了, 如果是替换, 则需要长度相等。如果发现不一样, 则跳过长的字符串对应的一个字符, 然后判断剩下的即可。

```
1 def oneEdit(stringOne, stringTwo):  
2     lengthOne, lengthTwo = len(stringOne), len(stringTwo)  
3  
4     if abs(lengthOne - lengthTwo) > 1:  
5         return False  
6  
7     for i in range(min(lengthOne, lengthTwo)):  
8         if stringOne[i] != stringTwo[i]:  
9             if lengthOne > lengthTwo:  
10                return stringOne[i+1:] == stringTwo[i:]  
11            elif lengthTwo > lengthOne:  
12                return stringOne[i:] == stringTwo[i+1:]  
13            else:  
14                return stringOne[i+1:] == stringTwo[i+1:]  
15  
16     return True  
17
```

Union Find

Union Find



Write a `UnionFind` class that implements the union-find (also called a disjoint set) data structure. This class should support three methods:

The union-find data structure is similar to a traditional set data structure in that it contains a collection of unique values. However, these values are spread out amongst a variety of distinct disjoint sets, meaning that no set can have duplicate values, and no two sets can contain the same value.

- `createSet(value)` : Adds a given value in a new set containing only that value.
- `union(valueOne, valueTwo)` : Takes in two values and determines which sets they are in. If they are in different sets, the sets are combined into a single set. If either value is not in a set or they are in the same set, the function should have no effect.
- `find(value)` : Returns the "representative" value of the set for which a value belongs to. This can be any value in the set, but it should always be the same value, regardless of which value in the set `find` is passed. If the value is not in a set, the function should return `null` / `None` . Note that after a set is part of a union, its representative can potentially change.

You can assume `createSet` will never be called with the same value twice.

If you're unfamiliar with Union Find, we recommend watching the Conceptual Overview section of this question's video explanation before starting to code.

Sample Usage

```
createSet(5): null
createSet(10): null
find(5): 5
find(10): 10
union(5, 10): null
find(5): 5
find(10): 5
createSet(20): null
find(20): 20
union(20, 10): null
find(5): 5
find(10): 5
find(20): 5
```

思路:通过字典设置对应的内置函数,理解题意即可。

```
1 # Do not edit the class below except for
2 # the constructor and the createSet, find,
3 # and union methods. Feel free to add new
4 # properties and methods to the class.
5 class UnionFind:
6     def __init__(self):
7         self.parents = {}
8
9     def createSet(self, value):
10        # Write your code here
11        self.parents[value] = value
12
13    def find(self, value):
14        # Write your code here
15        if value not in self.parents:
16            return None
17        currentParent = value
18        while currentParent != self.parents[currentParent]:
19            currentParent = self.parents[currentParent]
20        return currentParent
```



```

21     def union(self, valueOne, valueTwo):
22         # Write your code here
23         if valueOne not in self.parents or valueTwo not in self.parents:
24             return
25
26         valueOneRoot = self.find(valueOne)
27         valueTwoRoot = self.find(valueTwo)
28         self.parents[valueTwoRoot] = valueOneRoot

```

Four Number Sum

Four Number Sum 🟢 ☆

Write a function that takes in a non-empty array of distinct integers and an integer representing a target sum. The function should find all quadruplets in the array that sum up to the target sum and return a two-dimensional array of all these quadruplets in no particular order.

If no four numbers sum up to the target sum, the function should return an empty array.

Sample Input

```

array = [7, 6, 4, -1, 1, 2]
targetSum = 18

```

Sample Output

```
[[7, 6, 4, -1], [7, 6, 1, 2]] /
```

思路: #把四个数求和组合成2个数求和, 通过hash表进行查找difference。这里的i始终确保有左右2个部分。或者直接4个for循环即可。

```

1  def fourNumberSum(array, targetSum):
2      allPairSums = {}
3      quadruplets = []
4
5      #把四个数求和组合成2个数求和, 通过hash表进行查找difference。
6      #这里的i始终确保有左右2个部分
7      for i in range(1, len(array)-1):
8          for j in range(i+1, len(array)):
9              currentSum = array[i] + array[j]
10             difference = targetSum - currentSum

```

```

11         if difference in allPairSums:
12             for pair in allPairSums[difference]:
13                 quadruplets.append(pair + [array[i],array[j]])
14     for k in range(0,i):
15         currentSum = array[i] + array[k]
16         if currentSum not in allPairSums:
17             allPairSums[currentSum] = [[array[k],array[i]]]
18         else:
19             allPairSums[currentSum].append([array[k],array[i]])
20     print(allPairSums)
21     return quadruplets

```

Subarray Sort

Write a function that takes in an array of at least two integers and that returns an array of the starting and ending indices of the smallest subarray in the input array that needs to be sorted in place in order for the entire input array to be sorted (in ascending order).

If the input array is already sorted, the function should return `[-1, -1]`.

Sample Input

```
array = [1, 2, 4, 7, 10, 11, 7, 12, 6, 7, 16, 18, 19]
```

Sample Output

```
[3, 9]
```

思路:通过建立2个字典，分表保存对应元素的位置，第一个字典存储原先的位置，另一个字典存储排序好的位置。然后通过比较两个列表，将位置有变动的元素其对应的位置保存在changedList中，如果该列表不为空，左边为最小值，右边为最大值。

```

1  def subarraySort(array):
2      left_num = -1
3      right_num = -1
4
5      list1_dict = {}
6      list2_dict = {}
7      changed_list = []
8
9      for i in range(len(array)):
10         list1_dict[array[i]] = i

```

```

11     array.sort()
12     for i in range(len(array)):
13         list2_dict[array[i]] = i
14
15
16     for element_dict in list1_dict:
17         if list1_dict[element_dict] != list2_dict[element_dict]:
18             changed_list.append(list1_dict[element_dict])
19             changed_list.append(list2_dict[element_dict])
20
21     if len(changed_list) != 0:
22         left_num = min(changed_list)
23         right_num = max(changed_list)
24     return [left_num, right_num]

```

Subarray Sort

Sample Input

```
array = [1, 11, 3, 0, 15, 5, 2, 4, 10, 7, 12, 6]
```

Sample Output

```
[0, 7]
```

思路:可以在列表中找到的最大范围组合，图中就是0-7，每个数。所以整体的思路就是先把对应数组排好序，然后判断是否有中断，中断了则将对对应元素的距离和列表左右值存储在字典中，从字典中寻找到最大值即可。

```

1  def largestRange(array):
2      # Write your code here.
3
4      array.sort()
5      output_list = []
6      if array is None:
7          output_list = []
8      elif len(array) == 1:
9          output_list = [array[0], array[0]]
10
11     else:

```

```

11         length_dict = {}
12         begin_i = 0
13         for i in range(len(array)-1):
14             if array[i+1] != array[i] + 1 and array[i+1] != array[i]:
15                 length_dict[array[i]-array[begin_i]] = [array[begin_i], array[i]]
16                 begin_i = i + 1
17
18         length_dict[array[len(array)-1]-array[begin_i]] = [array[begin_i], array[len(array)-1]]
19
20         max_length = 0
21         for each_length in length_dict:
22             if max_length < each_length:
23                 max_length = each_length
24
25         output_list = length_dict[max_length]
26
27     return output_list

```

Min Rewards

Imagine that you're a teacher who's just graded the final exam in a class. You have a list of student scores on the final exam in a particular order (not necessarily sorted), and you want to reward your students. You decide to do so fairly by giving them arbitrary rewards following two rules:

1. All students must receive at least one reward.
2. Any given student must receive strictly more rewards than an adjacent student (a student immediately to the left or to the right) with a lower score and must receive strictly fewer rewards than an adjacent student with a higher score.

Write a function that takes in a list of scores and returns the minimum number of rewards that you must give out to students to satisfy the two rules.

You can assume that all students have different scores; in other words, the scores are all unique.

Sample Input

```
scores = [8, 4, 2, 1, 3, 6, 7, 9, 5]
```

Sample Output

```
25 // you would give out the following rewards: [4, 3, 2, 1, 2, 3, 4, 5, 1]
```

思路:当为单调减的时候, 需要确定出什么时候单调为止, 并且对前面的数进行回溯, 从而确定能给的最小数值。

当为单调增的时候, 就不用考虑这些了, 直接+1即可。

```
1 def minRewards(scores):
2     rewards = [1 for _ in scores]
3     for i in range(1, len(scores)):
4         j = i - 1
5         if scores[i] > scores[j]:
6             rewards[i] = rewards[j] + 1
7         else:
8             while j >= 0 and scores[j] > scores[j+1]:
9                 rewards[j] = max(rewards[j], rewards[j+1]+1)
10                j -= 1
11     return sum(rewards)
12
```

Zigzag Traverse

Zigzag Traverse

Write a function that takes in an $n \times m$ two-dimensional array (that can be square-shaped when $n == m$) and returns a one-dimensional array of all the array's elements in zigzag order.

Zigzag order starts at the top left corner of the two-dimensional array, goes down by one element, and proceeds in a zigzag pattern all the way to the bottom right corner.

Sample Input

```
array = [  
    [1, 3, 4, 10],  
    [2, 5, 9, 11],  
    [6, 8, 12, 15],  
    [7, 13, 14, 16],  
]
```

Sample Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
```

思路:首先解题思路分为2个步骤,一个是goingdown 一个并非goingdown,所以在进行边界判断后,确定还是在边界范围内,则进行对应的步骤操作,如果是向下传递,则先判断它是否在左边界处和下边界处(碰到边界后就会发生对应的转向机goingdown反向)。如果不在左边界处则可以对列进行-1和行进行+1操作。

```
1 def zigzagTraverse(array):  
2     height = len(array) - 1  
3     width = len(array[0]) - 1  
4     result = []  
5     row, col = 0, 0  
6     goingDown = True  
7     while not isOutOfBounds(row, col, height, width):  
8         result.append(array[row][col])  
9  
10        if goingDown:  
11            if col == 0 or row == height:  
12                goingDown = False  
13                if row == height:  
14                    col += 1  
15            else:  
16                row += 1  
17        else:  
18            row += 1  
19            col -= 1  
20    else:  
21        if row == 0 or col == width:  
22            goingDown = True  
23            if col == width:  
24                row += 1
```

```

25         else:
26             col += 1
27     else:
28         row -= 1
29         col += 1
30     return result
31
32 def isOutOfBounds(row,col,height,width):
33     return row < 0 or row > height or col < 0 or col > width

```

Same Bsts

An array of integers is said to represent the Binary Search Tree (BST) obtained by inserting each integer in the array, from left to right, into the BST.

Write a function that takes in two arrays of integers and determines whether these arrays represent the same BST. Note that you're *not* allowed to construct any BSTs in your code.

A BST is a Binary Tree that consists only of **BST** nodes. A node is said to be a valid **BST** node if and only if it satisfies the BST property: its value is strictly greater than the values of every node to its left; its value is less than or equal to the values of every node to its right; and its children nodes are either valid **BST** nodes themselves or **None** / **null**.

Sample Input

```

arrayOne = [10, 15, 8, 12, 94, 81, 5, 2, 11]
arrayTwo = [10, 8, 5, 15, 2, 12, 11, 94, 81]

```

Sample Output

```
true // both arrays represent the BST below
```



思路: 首先进行长度判断, 即通过长度来确定递归条件的结束, 然后分别对应列表的左子树和右子树进行判断筛选。

```

1 # Press Double Shift to search everywhere for classes, files, tool windows, actions, and
2 def sameBsts(arrayOne, arrayTwo):

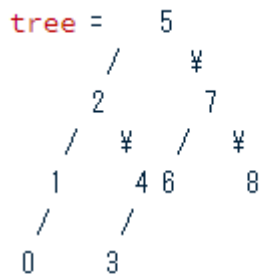
```

```

3     if len(arrayOne) != len(arrayTwo):
4         return False
5     if len(arrayOne) == 0 and len(arrayTwo) == 0:
6         return True
7
8     if arrayOne[0] != arrayTwo[0]:
9         return False
10
11    leftOne = getSmaller(arrayOne)
12    leftTwo = getSmaller(arrayTwo)
13    rightOne = getBiggerOrEqual(arrayOne)
14    rightTwo = getBiggerOrEqual(arrayTwo)
15
16    return sameBsts(leftOne, leftTwo) and sameBsts(rightOne, rightTwo)
17
18
19 def getSmaller(array):
20     smaller = []
21     for i in range(1, len(array)):
22         if array[i] < array[0]:
23             smaller.append(array[i])
24     return smaller
25
26
27 def getBiggerOrEqual(array):
28     biggerOrEqual = []
29     for i in range(1, len(array)):
30         if array[i] >= array[0]:
31             biggerOrEqual.append(array[i])
32     return biggerOrEqual
33

```

Validate Three Nodes



// This tree won't actually be passed as an input; it's here to help you visualize th

nodeOne = 5 // The actual node with value 5.

nodeTwo = 2 // The actual node with value 2.

nodeThree = 3 // The actual node with value 3.

思路:简而言之就是判断三个节点的单调性。是否一致为子节点，从前往后，或者从后往前。

```

1  # This is an input class. Do not edit.
2  class BST:
3      def __init__(self, value, left=None, right=None):
4          self.value = value
5          self.left = left
6          self.right = right
7
8
9  def validateThreeNodes(nodeOne, nodeTwo, nodeThree):
10     #如果nodeOne是nodeTwo的descendant则，验证，nodeTwo是否是nodeThree的descendant
11     if isDescendant(nodeTwo,nodeOne):
12         return isDescendant(nodeThree,nodeTwo)
13
14     #如果nodeThree是nodeTwo的descendant则，验证，nodeTwo是否是nodeOne的descendant
15     if isDescendant(nodeTwo,nodeThree):
16         return isDescendant(nodeOne,nodeTwo)
17
18     return False
19
20 def isDescendant(node,target):
21     if node is None:
22         return False
23     elif node is target:
24         return True
25
26     return isDescendant(node.left,target) if target.value < node.value else isDescendant(

```

Max Path Sum In Binary Tree

Write a function that takes in a Binary Tree and returns its max path sum.

A path is a collection of connected nodes in a tree, where no node is connected to more than two other nodes; a path sum is the sum of the values of the nodes in a particular path.

Each `BinaryTree` node has an integer `value`, a `left` child node, and a `right` child node. Children nodes can either be `BinaryTree` nodes themselves or `None` / `null`.

Sample Input

```
tree = 1
      /  \
     2    3
    / \  / \
   4  5 6  7
```

Sample Output

```
18 // 5 + 2 + 1 + 3 + 7
```

思路:给这个子树设计两个值,一个是当前节点作为分支的最大求和,另一个是当前节点的最大路径求和。当遍历到叶子节点后,返回一个(0, 最小负数)。

```
1 def maxPathSum(tree):
2     maxSumAsBranch, maxSum = findMaxSum(tree)
3     print(maxSumAsBranch)
4     return maxSum
5
6
7 def findMaxSum(tree):
8     if tree is None:
9         return (0, float("-inf"))
10
11     leftMaxSumAsBranch, leftMaxPathSum = findMaxSum(tree.left)
12     rightMaxSumAsBranch, rightMaxPathSum = findMaxSum(tree.right)
13     maxChildSumAsBranch = max(leftMaxSumAsBranch, rightMaxSumAsBranch)
14
15     value = tree.value
16     //子树的最大值加上当前节点的值和当前节点值大小进行比较。
17     maxSumAsBranch = max(maxChildSumAsBranch + value, value)
```

```

18 //将当前节点看做一个root节点，其当前节点能获得路径值为左子树+右子树+当前值，
19 //并和上面的值进行比较。
20 maxSumAsRootNode = max(leftMaxSumAsBranch+value+rightMaxSumAsBranch,maxSumAsBranch)
21 maxPathSum = max(leftMaxPathSum,rightMaxPathSum,maxSumAsRootNode)
22
23 return (maxSumAsBranch,maxPathSum)

```

Find Nodes Distance K

Sample Input

```

tree = 1
  /  \
 2    3
 / \  \
4  5  6
      / \
      7  8

target = 3
k = 2

```

Sample Output

```
[2, 7, 8] // These values could be ordered differently.
```

思路：注释写得很详细，看注释就好了。

```

1 # This is an input class. Do not edit.
2 class BinaryTree:
3     def __init__(self, value, left=None, right=None):
4         self.value = value
5         self.left = left
6         self.right = right
7
8
9 def findNodesDistanceK(tree, target, k):
10     #得到父节点字典。
11     parentDict = {}
12     getParentDict(tree,parentDict,parent=None)
13     print(parentDict)
14
15     #得到子节点的字典

```

```

16     childDict = {}
17     getChildDict(tree, childDict)
18     print(childDict)
19
20     #给所有节点设置一个标志位字典，防止面递归查找重复递归的可能
21     flagDict = {}
22     getFlagDict(tree, flagDict)
23     print(flagDict)
24
25     #首先需要知道target在哪里，找到target。将target节点作为出发节点。
26     #首先查询父节点的字典，并将父节点和该节点的距离设置为1
27     #查询target节点的子节点，并将其的距离设置为1
28     #然后逐步递归找到等于k的值，并将该值添加到输出函数之中。
29
30     kDistanceNodeList = []
31     findKdistanceNode(parentDict, childDict, kDistanceNodeList, flagDict, target, k)
32     return kDistanceNodeList
33 def getParentDict(node, parentDict, parent = None):
34     if node is not None:
35         parentDict[node.value] = parent
36         getParentDict(node.left, parentDict, node)
37         getParentDict(node.right, parentDict, node)
38
39 def getChildDict(node, childDict):
40     if node is not None:
41         childDict[node.value] = [node.left, node.right]
42         getChildDict(node.left, childDict)
43         getChildDict(node.right, childDict)
44
45
46 def getFlagDict(node, flagDict):
47     if node is not None:
48         flagDict[node.value] = True
49         getFlagDict(node.left, flagDict)
50         getFlagDict(node.right, flagDict)
51
52 # 找到距离目标距离为k的节点，根据parentDict和childDict。
53
54 def findKdistanceNode(parentDict, childDict, kDistanceNodeList, flagDict, currentNodeValue, k

```

```

55     #作为结束的条件
56     if flagDict[currentNodeValue]:
57         #首先找到target节点其对应的子节点和父节点
58         parentNode = parentDict[currentNodeValue]
59         childNode = childDict[currentNodeValue]
60         flagDict[currentNodeValue] = False
61         if parentNode is not None:
62             findKdistanceNode(parentDict, childDict, kDistanceNodeList, flagDict, parentNode)
63
64         if childNode[0] is not None and flagDict[childNode[0].value]:
65             findKdistanceNode(parentDict, childDict, kDistanceNodeList, flagDict, childNode[0])
66         if childNode[1] is not None and flagDict[childNode[1].value]:
67             findKdistanceNode(parentDict, childDict, kDistanceNodeList, flagDict, childNode[1])
68
69         if k==0:
70             kDistanceNodeList.append(currentNodeValue)

```

Max Sum Increasing Subsequence

Sample Input

```
array = [10, 70, 20, 30, 50, 11, 30]
```

Sample Output

```
[110, [10, 20, 30, 50]] // The subsequence [10, 20, 30, 50] is strictly increasing and has the maximum sum.
```

思路:注释写得很详细,方法也是非常常见的一种动态规划的一种,通过2个for循环,不断获取当前位置的最优解。一般最后一位就表示整个数列的最优解,但本题还有一个变通就在于最后的数列构建。

```

1 def maxSumIncreasingSubsequence(array):
2     #通过构造一个sum_array 列表来显示 increase条件下,且为正数时,的各个位置的最大值
3     #找到其中最大的一个位置的值。
4     sum_array = [0 for _ in array]
5     for i in range(len(array)):
6         currentNum = array[i]
7         for j in range(i+1, len(array)):
8             if array[i] < array[j] and sum_array[j] < sum_array[i] + currentNum:
9                 sum_array[j] = sum_array[i] + currentNum

```

```

10
11
12     max_nums = max(sum_array)
13     subsequences = []
14     constructList(max_nums, array, sum_array, subsequences)
15     subsequences = list(reversed(subsequences))
16
17     return [max_nums, subsequences]
18
19     #通过最大的值和其对应的位置上的array值不断循环构建序列。
20     #如果最大的值等于array值,则退出循环, 输出当前构建的序列。
21
22
23
24 def constructList(max_nums, array, sum_array, subsequences = []):
25     #找到max_nums的位置
26     locate_max_num = 0
27     #注意, 这里需要从后往前遍历。
28     for i in reversed(range(len(sum_array))):
29         if sum_array[i] == max_nums:
30             locate_max_num = i
31             break
32     #然后判断该位置上的array值是否等于max_nums值, 等于的话就返回列表
33     if array[locate_max_num] == max_nums:
34         subsequences.append(array[locate_max_num])
35         return
36     #如果不等于, 则令最大值减去array值, 并将array值添加进subsequences中, 并重新循环
37     elif max_nums > array[locate_max_num]:
38         subsequences.append(array[locate_max_num])
39         max_nums -= array[locate_max_num]
40         constructList(max_nums, array, sum_array, subsequences)

```

Longest Common Subsequence

Sample Input

```
str1 = "ZXVVYZW"  
str2 = "XKYKZPW"
```

Sample Output

```
["X", "Y", "Z", "W"]
```

这是一个使用动态规划求解两个字符串最长公共子序列 (Longest Common Subsequence, LCS) 的 Python 函数。下面是对这个函数的逐行分析：

1. 初始化一个二维列表 `lcs`，大小为 $(\text{len}(\text{str2}) + 1) \times (\text{len}(\text{str1}) + 1)$ 。这个二维列表将用于存储动态规划的中间结果。`lcs[i][j]` 表示 `str2` 的前 `i` 个字符和 `str1` 的前 `j` 个字符的最长公共子序列。
2. 使用两层嵌套循环遍历 `lcs` 列表的每个元素。外层循环遍历 `str2`，内层循环遍历 `str1`。
3. 如果当前字符 `str2[i-1]` 等于 `str1[j-1]`，则 `lcs[i][j]` 等于 `lcs[i-1][j-1]` 加上当前字符。这意味着，如果当前字符相同，那么它们一定在最长公共子序列中。
4. 如果当前字符不相同，那么 `lcs[i][j]` 等于 `lcs[i-1][j]` 和 `lcs[i][j-1]` 中较长的一个。这是因为我们需要找到一个最长的公共子序列，所以在这种情况下，我们选择两个子问题中最优的一个。
5. 在遍历过程中，打印出 `lcs` 列表。这可以帮助我们观察算法的执行过程。
6. 返回 `lcs[-1][-1]`，即整个 `str2` 和 `str1` 的最长公共子序列。这是我们要求的结果。

这个函数的时间复杂度和空间复杂度都是 $O(\text{len}(\text{str1}) * \text{len}(\text{str2}))$ ，因为它使用了一个二维列表来存储动态规划的结果，并且需要遍历所有可能的字符组合。

```
1 def longestCommonSubsequence(str1, str2):  
2     lcs = [[[] for x in range(len(str1)+1)] for y in range(len(str2)+1)]  
3     for i in range(1, len(str2)+1):  
4         for j in range(1, len(str1) + 1):  
5             if str2[i-1] == str1[j-1]:  
6                 lcs[i][j] = lcs[i-1][j-1] + [str2[i-1]]  
7             else:  
8                 lcs[i][j] = max(lcs[i-1][j], lcs[i][j-1], key = len)  
9     print(lcs)  
10    return lcs[-1][-1]
```

Count Squares

```
points = [
    [1, 1],
    [0, 0],
    [-4, 2],
    [-2, -1],
    [0, 1],
    [1, 0],
    [-1, 4]
]
```

Sample Output

```
// [1, 1], [0, 0], [0, 1], and [1, 0] makes a square,
2 // as does [1, 1], [-4, 2], [-2, -1], and [-1, 4]
```

min Number Of Jumps

You're given a non-empty array of positive integers where each integer represents the maximum number of steps you can take forward in the array. For example, if the element at index `1` is `3`, you can go from index `1` to index `2`, `3`, or `4`.

Write a function that returns the minimum number of jumps needed to reach the final index.

Note that jumping from index `i` to index `i + x` always constitutes one jump, no matter how large `x` is.

Sample Input

```
array = [3, 4, 2, 1, 2, 3, 7, 1, 1, 1, 3]
```

Sample Output

```
4 // 3 → (4 or 2) → (2 or 3) → 7 → 3
```

思路：还是动态规划，通过辅助列表求解每个对应数字的最小jump即可。

```
1 def minNumberOfJumps(array):
2     #验证jumps时通过辅助列表jumps可以求得各个元素的最小jumps并保存。
3     jumps = [float("inf") for x in array]
4     jumps[0] = 0
5
6     for i in range(1, len(array)):
7         for j in range(0, i):
8             if array[j] + j >= i:
9                 jumps[i] = min(jumps[j] + 1, jumps[i])
10
11     print(jumps)
12     return jumps[-1]
```

water Area

Sample Input

```
heights = [0, 8, 0, 0, 5, 0, 0, 10, 0, 0, 1, 1, 0, 3]
```

Sample Output

48

```
// Below is a visual representation of the sample input.
// The dots and vertical lines represent trapped water and pillars, respectively.
// Note that there are 48 dots.
//      |
//      |
// |.....|
// |.....|
// |.....|
// |..|..|
// |..|..|
// |..|..|.....|
// |..|..|.....|
// _|..|..|..||.|
```

Test Case 1

```
[0, 0, 8, 8, 8, 8, 8, 8, 10, 10, 10, 10, 10, 10]
[0, 0, 8, 8, 3, 8, 8, 0, 3, 3, 2, 2, 3, 0]
```

分析：动态规划问题。获得一个左边最大值的数组，然后和右边进行比较取最小值，这就是对应的容纳面积。两处print的结果，方便理解分析。

```
1 def waterArea(heights):
2     maxes= [0 for x in heights]
3     leftMax = 0
4     for i in range(len(heights)):
5         height = heights[i]
6         maxes[i] = leftMax
7         leftMax = max(leftMax,height)
8     rightMax = 0
9     print(maxes)
10    for i in reversed(range(len(heights))):
11        height = heights[i]
12        minHeight = min (rightMax,maxes[i])
13
14        if height < minHeight:
15            maxes[i] = minHeight - height
```

```

16         else:
17             maxes[i] = 0
18
19             rightMax = max(rightMax,height)
20     print(maxes)
21     return sum(maxes)

```

knapsack Problem

Sample Input

```

items = [[1, 2], [4, 3], [5, 6], [8, 7]]
capacity = 10

```

Sample Output

```

[10, [1, 3]] // items [4, 3] and [8, 7]

```

思路:这段代码实现了经典的背包问题（Knapsack Problem）的解决方案。动态规划问题。

函数**knapsackProblem**接受两个参数：**items**和**capacity**。其中，**items**是一个二维列表，表示可供选择的物品，每个物品有对应的价值和重量。**capacity**表示背包的容量。

首先，代码创建了一个二维数组**knapsackValues**，用于存储每个物品放入背包时可能获得的最大价值。数组的行数为**items**的长度加1，列数为**capacity+1**。初始时，所有格子的值都为0。

接下来，使用两层循环遍历每个物品和背包容量。对于当前物品，获取其价值和重量，然后根据当前重量与背包容量的关系进行判断：

- 如果当前物品的重量大于当前背包容量，说明该物品无法放入背包，因此当前格子的值等于上一个物品在相同容量下的最大值。
- 否则，当前格子的值为上一个物品在相同容量下的最大值和放入当前物品后的最大值中的较大者。放入当前物品后的最大值通过计算
 - `knapsackValues[i-1][c - currentWeight] + currentValue`
 - 得到，其中
 - `knapsackValues[i-1][c - currentWeight]`
 - 表示在剩余容量下放入当前物品的最大值，
 - `currentValue`
 - 表示当前物品的价值。

完成循环后，**knapsackValues[-1][-1]**表示背包容量为**capacity**时能够获得的最大总价值。

函数**getKnapsackItems**用于获取被选中的物品序列。它接受**knapsackValues**和**items**作为参数。通过追踪背包的填充过程，从最后一个格子开始，逐步向前确定哪些物品被选中放入背包。具体步骤如下：

- 初始化空序列sequence。从最后一个物品开始，依次向前遍历每个物品。如果当前物品的价值与上一个物品在相同容量下的最大值相等，说明当前物品没有被选中，将索引i-1添加到序列中，并更新剩余容量c和物品索引i。否则，说明当前物品被选中，将索引i-1添加到序列中，并更新剩余容量c和物品索引i。如果剩余容量c等于0，表示背包已经填满，跳出循环。返回反转后的序列作为结果。

最终，函数knapsackProblem返回一个包含两个元素的列表，第一个元素是背包容量为capacity。

```
1 def knapsackProblem(items, capacity):
2     knapsackValues = [[0 for x in range(0, capacity + 1)] for y in range(0, len(items) + 1)]
3     print(knapsackValues)
4
5     for i in range(1, len(items) + 1):
6         currentWeight = items[i - 1][1]
7         currentValue = items[i - 1][0]
8         for c in range(0, capacity + 1):
9             if currentWeight > c:
10                 knapsackValues[i][c] = knapsackValues[i - 1][c]
11             else:
12                 knapsackValues[i][c] = max(
13                     knapsackValues[i - 1][c], knapsackValues[i - 1][c - currentWeight] + currentValue
14                 )
15     return [knapsackValues[-1][-1], getKnapsackItems(knapsackValues, items)]
16
17 def getKnapsackItems(knapsackValues, items):
18     sequence = []
19     i = len(knapsackValues) - 1
20     c = len(knapsackValues[0]) - 1
21
22     while i > 0:
23         if knapsackValues[i][c] == knapsackValues[i - 1][c]:
24             i -= 1
25         else:
26             sequence.append(i - 1)
27             c -= items[i - 1][1]
28             i -= 1
29
30         if c == 0:
31             break
32     return list(reversed(sequence))
```

Disk Stacking

Sample Input

```
disks = [[2, 1, 2], [3, 2, 3], [2, 2, 8], [2, 3, 4], [1, 3, 1], [4, 4, 5]]
```

Sample Output

```
[[2, 1, 2], [3, 2, 3], [4, 4, 5]]
// 10 (2 + 3 + 5) is the tallest height we can get by
// stacking disks following the rules laid out above.
```

思路:这段代码实现了一个问题，即给定一组圆盘的尺寸（宽度、深度、高度），要求在堆叠这些圆盘时，找出最大高度的堆叠序列。

函数**diskStacking**接受一个圆盘列表**disks**作为输入。首先，它对圆盘列表进行排序，按照高度（**disk[2]**）进行升序排序。然后，创建一个列表**heights**，用于存储每个圆盘堆叠时的最大高度。另外，创建一个列表**sequences**，用于记录每个圆盘的堆叠序列。

接下来，定义变量**maxHeightIdx**，用于记录当前已经计算过的最大高度的圆盘索引。

通过两层循环遍历每个圆盘。对于当前圆盘，遍历之前的圆盘，并判断是否满足堆叠条件（通过函数**areValidDimensions**判断）。如果满足条件，并且通过堆叠能够获得更高的高度，则更新**heights[i]**为当前圆盘高度加上之前堆叠圆盘的最大高度，并更新**sequences[i]**为之前的圆盘索引。

每次计算完当前圆盘后，更新**maxHeightIdx**的值为当前最大高度的圆盘索引。

最后，打印输出调试信息，包括圆盘列表、高度列表、序列列表和最大高度的圆盘索引。然后，调用函数**buildSequence**根据最大高度的圆盘索引构建堆叠序列，并返回反转后的序列作为结果。

函数**areValidDimensions**用于判断两个圆盘是否满足堆叠条件，即当前圆盘的宽度、深度和高度均小于之前圆盘的对应尺寸。

函数**buildSequence**根据堆叠序列**sequences**和最大高度的圆盘索引构建堆叠序列。从最大高度的圆盘开始，依次向前遍历序列，将每个圆盘添加到结果序列中。最后，返回反转后的序列作为结果。

```
1 def diskStacking(disks):
2     #[width,depth,height]将disk按照高度排序
3     disks.sort(key = lambda disk:disk[2])
4     heights = [disk[2] for disk in disks]
5     sequences = [None for disk in disks]
6
7     maxHeightIdx = 0
8
9     for i in range(1,len(disks)):
10         currentDisk = disks[i]
```

```

11         for j in range(0,i):
12             otherDisk = disks[j]
13             if areValidDimensions(otherDisk,currentDisk):
14                 if heights[i] <= currentDisk[2] + heights[j]:
15                     heights[i] = currentDisk[2] + heights[j]
16                     sequences[i] = j
17             #每次算完更新一次maxHeightIdx的坐标
18             if heights[i] >= heights[maxHeightIdx]:
19                 maxHeightIdx = i
20
21     print(disks)
22     print(heights)
23     print(sequences)
24     print(maxHeightIdx)
25
26     return buildSequence(disks,sequences,maxHeightIdx)
27
28 def areValidDimensions(o,c):
29     return o[0] < c[0] and o[1] < c[1] and o[2] < c[2]
30
31 def buildSequence(array,sequences,currentIdx):
32     sequence = []
33     while currentIdx is not None:
34         sequence.append(array[currentIdx])
35         currentIdx = sequences[currentIdx]
36     return list(reversed(sequence))

```

numbers In Pi

Sample Input

```

pi = "3141592653589793238462643383279",
numbers = ["314159265358979323846", "26433", "8", "3279", "314159265", "358979323846"]

```

Sample Output

```

2 // "314159265 | 35897932384626433832 | 79"

```

思路:

这段代码实现了一个函数 `numbersInPi(pi, numbers)`，它的目标是找到将给定的 π (pi) 字符串拆分为由给定数字组成的最小空格数。下面是对代码的逐行解释：

python

 Copy code

```
def numbersInPi(pi, numbers):
    numbersTable = {number: True for number in numbers}
    print(numbersTable)
    minSpaces = getMinSpaces(pi, numbersTable, {}, 0)
    return -1 if minSpaces == float("inf") else minSpaces
```

这是主函数 `numbersInPi`，它接受两个参数：`pi` 是表示 π 的字符串，`numbers` 是一个数字列表。代码首先创建一个字典 `numbersTable`，其中包含了 `numbers` 列表中的数字作为键，并将其值设置为 `True`。然后，通过调用 `getMinSpaces` 函数来计算将 `pi` 拆分为数字的最小空格数。最后，如果最小空格数为正无穷大，则返回 -1，否则返回最小空格数。

这是一个递归函数 `getMinSpaces`，用于计算将 `pi` 字符串从索引 `idx` 开始拆分为数字的最小空格数。它使用了动态规划的思想，并通过缓存 `cache` 来避免重复计算。具体来说，函数根据以下步骤进行计算：

- 如果 `idx` 等于 `pi` 的长度，表示已经处理完整个字符串，返回 -1。
- 如果 `idx` 在缓存 `cache` 中存在，则直接返回缓存中的结果，避免重复计算。
- 初始化 `minSpaces` 为正无穷大。
- 从 `idx` 开始遍历到字符串的末尾，依次取出子字符串 `prefix`。
- 如果 `prefix` 在 `numbersTable` 中存在，表示可以将 `prefix` 视为一个数字。
- 递归调用 `getMinSpaces` 函数处理剩余的后缀部分，得到 `minSpacesInSuffix`，即后缀部分的最小空格数。
- 更新 `minSpaces` 为当前 `prefix` 的空格数加上后缀部分的最小空格数的较小值。
- 将 `minSpaces` 存储到缓存 `cache` 中，以便下次使用。
- 返回 `minSpaces`。

```
1 def numbersInPi(pi, numbers):
2     numbersTable = {number: True for number in numbers}
3     print(numbersTable)
4     minSpaces = getMinSpaces(pi, numbersTable, {}, 0)
5
6     return -1 if minSpaces == float("inf") else minSpaces
7
8 def getMinSpaces(pi, numbersTable, cache, idx):
```

```

9     if idx == len(pi):
10         return -1
11
12     if idx in cache:
13         return cache[idx]
14
15     minSpaces = float("inf")
16
17     for i in range(idx, len(pi)):
18         prefix = pi[idx : i+1]
19         if prefix in numbersTable:
20             minSpacesInSuffix = getMinSpaces(pi, numbersTable, cache, i+1)
21             minSpaces = min(minSpaces, minSpacesInSuffix + 1)
22
23     cache[idx] = minSpaces
24
25     return cache[idx]

```

maximum Sum Submatrix

```

[
  [2, 4],
  [5, 6],
  [-3, 2],
]

```

If `size = 2`, then the 2x2 submatrices to consider are:

```

[2, 4]
[5, 6]
-----
[5, 6]
[-3, 2]

```

思路:看注释吧。

```

1 def maximumSumSubmatrix(matrix, size):
2     # Write your code here.
3     #首先我要知道matrix的row和col
4     row = len(matrix)
5     col = len(matrix[0])
6

```

```

7     print(row)
8     print(col)
9     #行的遍历是根据row - size 来判断。当size为1 循环4次。当size是3是，从行来看只能循环到行1.
10    #列的遍历也是根据size 来判断。
11
12    #给maxSum 赋予一个极小值
13    maxSum = float("-inf")
14    list_begin = []
15    for i in range(0,row - size +1):
16        for j in range(0,col - size +1):
17            list_begin.append([i,j])
18            tempSum = calculateSum(i,j,matrix,size)
19            if tempSum > maxSum:
20                maxSum = tempSum
21    print(list_begin)
22    return maxSum
23
24 def calculateSum(i,j,matrix,size):
25     tempSum = 0
26     for row in range(i,i+size):
27         for col in range(j,j+size):
28             tempSum += matrix[row][col]
29
30     return tempSum

```

maximize Expression

Sample Input

```
array = [3, 6, 1, -3, 2, 7]
```

Sample Output

```

4
// Choose a = 1, b = 3, c = 4, and d = 5
// -> 6 - (-3) + 2 - 7 = 4

```

```
1 def maximizeExpression(array):
```

```
2     if len(array) < 4:
```



```

3         return 0
4
5     #看成4个循环相加的问题。
6     #选最大值范围为0,len(array)-4.并选择出该位置的最大点
7     maxmizeFunction = float("-inf")
8     for idx1 in range(0,len(array)-3):
9         for idx2 in range(idx1+1,len(array)-2):
10            for idx3 in range(idx2+1,len(array)-1):
11                for idx4 in range(idx3+1,len(array)):
12                    tempFunction = array[idx1]-array[idx2]+array[idx3]-ar
13                    if tempFunction > maxmizeFunction:
14                        maxmizeFunction = tempFunction
15     return maxmizeFunction

```

juice Bottling

Sample Input

```
prices = [0, 1, 3, 2]
```

Sample Output

```

// We have 3 total units of juice,
// because the length of prices is 4.
// To maximize revenue, we split the juice into
[1, 2] // quantities of 1 and 2, giving a revenue of 1 + 3 = 4

```

思路:动态规划问题, 通过建立一个最大利润数组, 不断循环前面的数组, 得到最大利润。

```

1 def juiceBottling(prices):
2     # Write your code here.
3     numSizes = len(prices)
4     maxProfit = [0] * numSizes
5     solutions = [[]] * numSizes
6
7     for size in range(numSizes):
8         for dividingPoint in range(size + 1):
9             possibleProfit = maxProfit[size - dividingPoint] + prices[dividingPoint]
10            if possibleProfit > maxProfit[size]:
11                maxProfit[size] = possibleProfit
12                solutions[size] = [dividingPoint] + solutions[size - dividingPoint]
13     print(solutions)

```

14

```
return solutions[numSizes - 1]
```